

Intro2SE - Design - 7

MeoArc

*The student team is required to complete the **Design Document** for the assigned course project, following the attached template.*



Software Engineering Department
Faculty of Information and Technology
VNU-HCMUS

Table of Contents

1 Member Contribution Assessment	2
2 Conceptual Model	21
3 Architectural Design	23
1.1 Architecture Diagram	23
1.2 Class Diagram	24
1.3 Class Specifications	26
1.3.1 Class “User”	26
1.3.2 Class “Conversation”	27
1.3.3 Class “Message”	27
1.3.4 Class “ToolCall”	28
1.3.5 Class “ConfirmationRequest”	29
1.3.6 Class “Tool”	29
1.3.7 Class “Skill”	30
1.3.8 Class “TriageSkill”	31
1.3.9 Class “Email”	32
1.3.10 Class “EmailDraft”	33
4 Data Design	35
1.4 Data Diagram	35
1.5 Data Specification	36
5 User Interface and User Experience Design	59
1.6 Screen Diagram	59
1.7 Screen Specifications	61
1.7.1 Screen “Landing Page”	61
1.7.2 Screen “Login & Connect Account”	66
1.7.3 Screen “Main screen (Chat + Dynamic Canvas)”	67
1.7.4 Screen “Confirmation Dialog (Human-in-the-loop)”	68
1.7.5 Screen “Email List”	69
1.7.6 Screen “Settings Panel”	70
1.7.7 Screen “Conversation History List”	71
6 AI Usage Declaration	73
7 Presentation	74
8 Reflective Report	75

Software Design

Objectives

This document focus on the following topics:

- ✓ Completing the Software Design Document with the following contents:
 - Conceptual Model
 - Architectural Design
 - Data Design
 - User Interface Design •
- ✓ Understanding the Software Design Document.
- ✓ This document will be used as input for AI tools to verify the quality of subsequent project artifacts.

All project artifacts must remain consistent and synchronized.

For example, if the project proposal is modified during this phase, a new version of the proposal must be documented.

By the conclusion of the project, all versions of every artifact must be submitted to demonstrate the evolution of your work.

1 Member Contribution Assessment

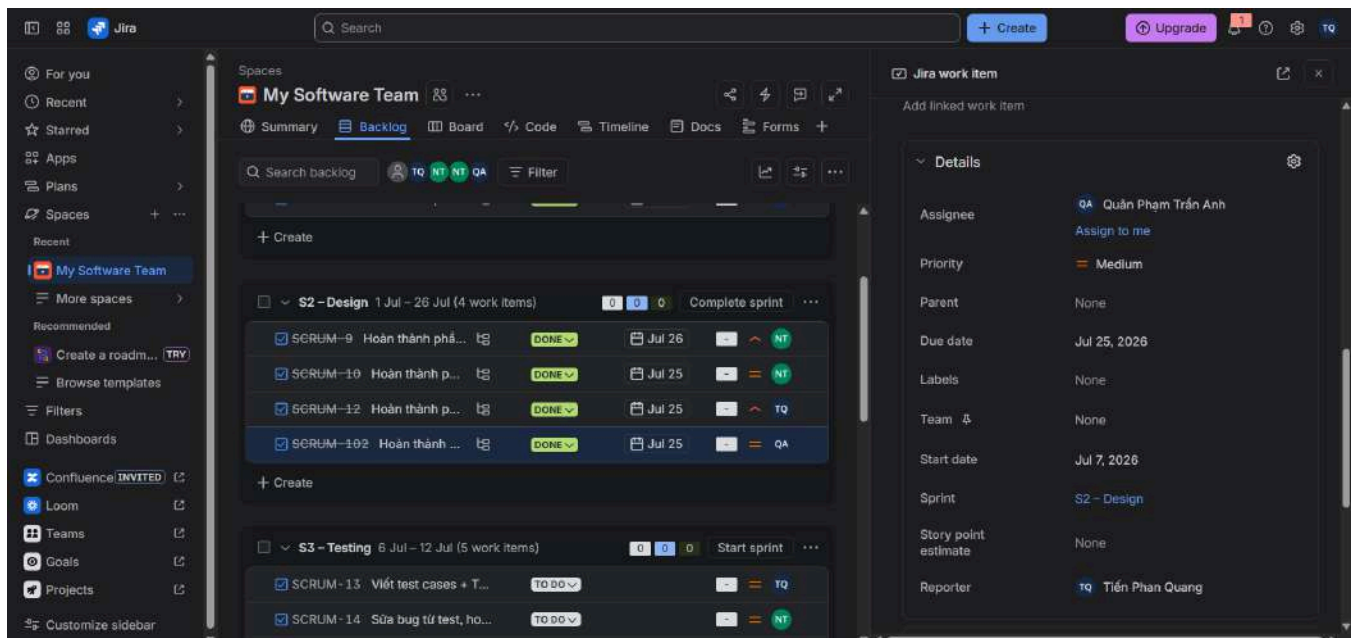
Written by: 24127250 - Phan Quang Tiến

Edited by: 24127226 - Phạm Trần Anh Quân

Reviewed by: 24127545 - Nguyễn Ngọc Thiên

24127226 - Phạm Trần Anh Quân - 25%

- Task Evidence:



The screenshot shows the Jira interface for 'My Software Team'. The main view is the 'Backlog' tab, displaying a list of work items under two sprints: 'S2 - Design' (1 Jul - 26 Jul) and 'S3 - Testing' (6 Jul - 12 Jul). The right sidebar shows the 'Edit mục 1 - Member Contribution' work item (SCRUM-102 / SCRUM-103). The work item details include:

- Assignee:** QA Quân Phạm Trần Anh
- Labels:** None
- Parent:** SCRUM-102 Hoàn thành phần đượ

The screenshot shows the Jira interface for 'My Software Team'. The main view is the 'Backlog' tab, displaying a list of work items under two sprints: 'S2 - Design' (1 Jul - 26 Jul) and 'S3 - Testing' (6 Jul - 12 Jul). The right sidebar shows the 'Review mục 2 - Conceptual Model' work item (SCRUM-102 / SCRUM-104). The work item details include:

- Assignee:** QA Quân Phạm Trần Anh
- Labels:** None
- Parent:** SCRUM-102 Hoàn thành phần đượ

The screenshot shows the Jira interface for the 'My Software Team' space. The main view is the 'Backlog' tab, displaying a list of work items. The right-hand panel shows the details for a specific work item, 'Review phần 1.1. Architecture Diagram' (SCRUM-102 / SCRUM-105).

Backlog Items:

Sprint	Item ID	Item Name	Status	Due Date	Assignee
S2 - Design (1 Jul - 26 Jul)	SCRUM-9	Hoàn thành ph...	DONE	Jul 26	NT
	SCRUM-10	Hoàn thành p...	DONE	Jul 25	NT
	SCRUM-12	Hoàn thành p...	DONE	Jul 25	TQ
	SCRUM-102	Hoàn thành ...	DONE	Jul 25	QA
S3 - Testing (8 Jul - 12 Jul)	SCRUM-13	Viết test cases + T...	TO DO		TQ
	SCRUM-14	Sửa bug từ test, ho...	TO DO		NT

Work Item Details (SCRUM-102 / SCRUM-105):

- Title:** Review phần 1.1. Architecture Diagram
- Description:** Add a description...
- Assignee:** QA Quân Phạm Trần Anh
- Labels:** None
- Parent:** SCRUM-102 Hoàn thành phần đượ

The screenshot shows the Jira interface for the 'My Software Team' space. The main view is the 'Backlog' tab, displaying a list of work items. The right-hand panel shows the details for a specific work item, 'Review phần 1.2. Class Diagram' (SCRUM-102 / SCRUM-106).

Backlog Items:

Sprint	Item ID	Item Name	Status	Due Date	Assignee
S2 - Design (1 Jul - 26 Jul)	SCRUM-9	Hoàn thành ph...	DONE	Jul 26	NT
	SCRUM-10	Hoàn thành p...	DONE	Jul 25	NT
	SCRUM-12	Hoàn thành p...	DONE	Jul 25	TQ
	SCRUM-102	Hoàn thành ...	DONE	Jul 25	QA
S3 - Testing (8 Jul - 12 Jul)	SCRUM-13	Viết test cases + T...	TO DO		TQ
	SCRUM-14	Sửa bug từ test, ho...	TO DO		NT

Work Item Details (SCRUM-102 / SCRUM-106):

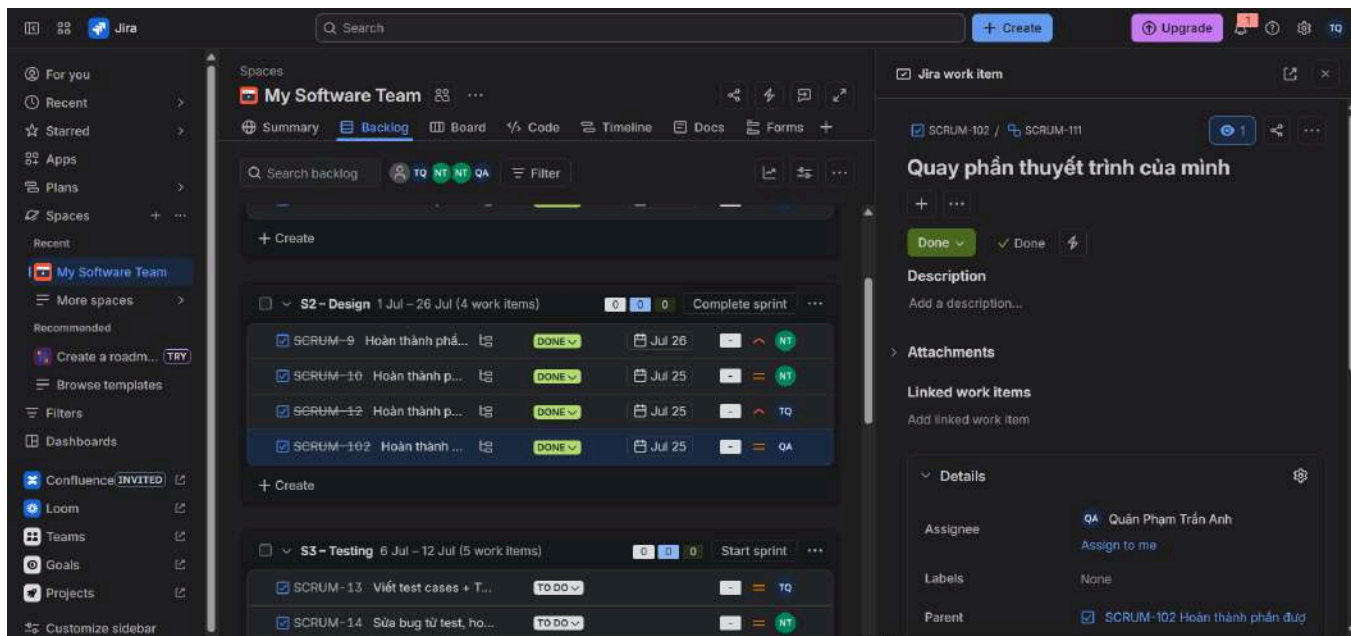
- Title:** Review phần 1.2. Class Diagram
- Description:** Add a description...
- Assignee:** QA Quân Phạm Trần Anh
- Labels:** None
- Parent:** SCRUM-102 Hoàn thành phần đượ

The screenshot displays the Jira interface for the 'My Software Team' space. The left sidebar shows navigation options like 'For you', 'Recent', 'Starred', 'Apps', 'Plans', 'Spaces', and 'Dashboards'. The main area shows the 'Backlog' view with a search bar and filters. Two sprints are visible: 'S2 - Design' (1 Jul - 26 Jul) and 'S3 - Testing' (6 Jul - 12 Jul). The 'S2 - Design' sprint contains four work items (SCRUM-9, SCRUM-10, SCRUM-12, SCRUM-102) all marked as 'DONE'. The 'S3 - Testing' sprint contains two work items (SCRUM-13, SCRUM-14) marked as 'TO DO'. On the right, a detailed view of a Jira work item is shown, titled 'Review phần 1.3 - Class Specification'. It includes a 'Done' button, a 'Description' field, and a 'Details' section with fields for 'Assignee' (QA Quân Phạm Trần Anh), 'Labels' (None), and 'Parent' (SCRUM-102 Hoàn thành phần đượ).

This screenshot shows the Jira interface for the 'My Software Team' space, similar to the one above. The main area displays the 'Backlog' view with the same sprint information. The detailed view on the right is for a Jira work item titled 'Review phần 1.4 - Data Design'. It includes a 'Done' button, a 'Description' field, and a 'Details' section with fields for 'Assignee' (QA Quân Phạm Trần Anh), 'Labels' (None), and 'Parent' (SCRUM-102 Hoàn thành phần đượ).

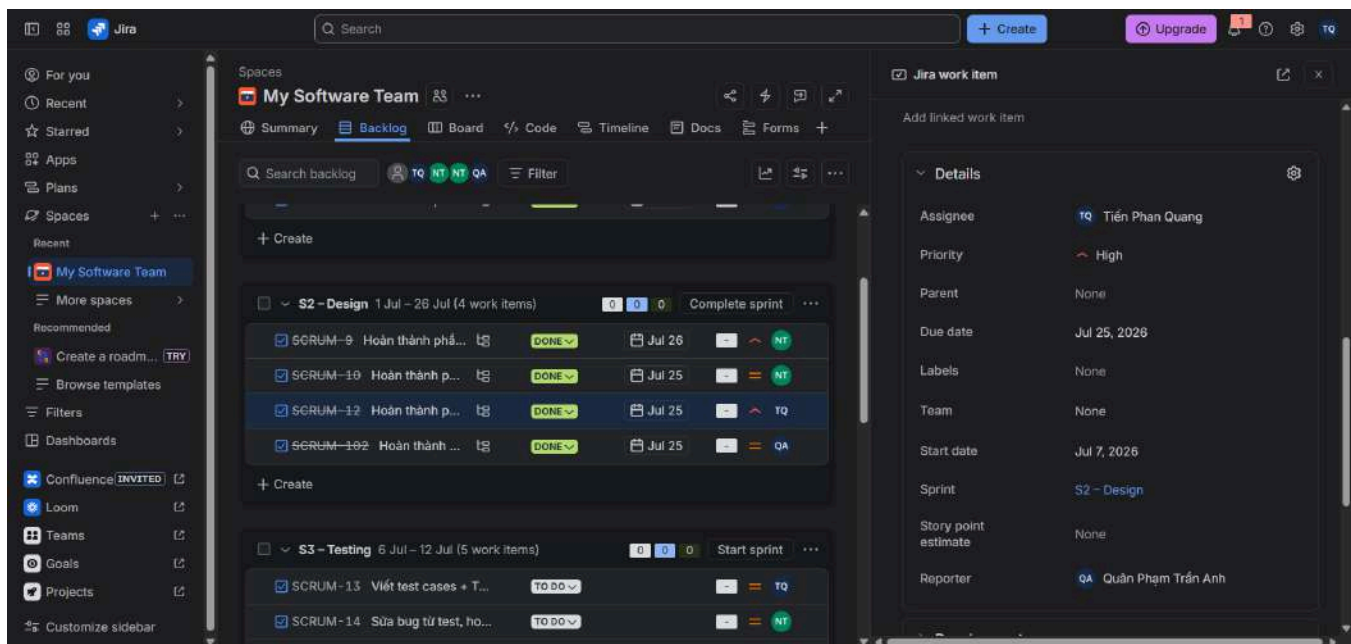
The screenshot displays the Jira interface for the 'My Software Team' space. The main view is the 'Backlog' tab, showing a list of work items organized into sprints. The 'S2 - Design' sprint (1 Jul - 26 Jul) is currently active and contains four items: SCRUM-9, SCRUM-10, SCRUM-12, and SCRUM-102, all of which are marked as 'DONE'. The 'S3 - Testing' sprint (6 Jul - 12 Jul) is also visible and contains two items: SCRUM-13 and SCRUM-14, both marked as 'TO DO'. The right sidebar provides details for the selected item, SCRUM-102, titled 'Review mục 4 - Data Design'. It shows the assignee as 'Quản Phạm Trần Anh' and the parent item as 'SCRUM-102: Hoàn thành phần 4'.

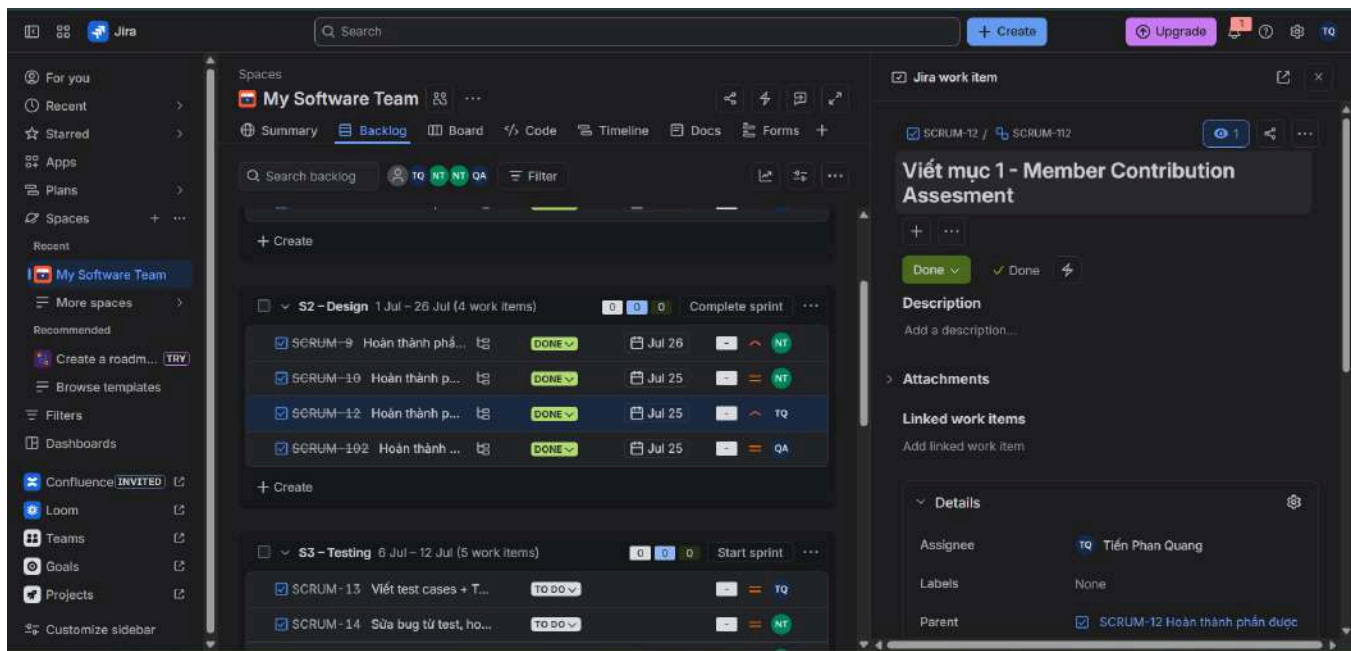
This screenshot shows the Jira interface for the 'My Software Team' space, similar to the one above. The 'Backlog' tab is active, showing the same two sprints: 'S2 - Design' (1 Jul - 26 Jul) and 'S3 - Testing' (6 Jul - 12 Jul). The 'S2 - Design' sprint contains four items: SCRUM-9, SCRUM-10, SCRUM-12, and SCRUM-102, all marked as 'DONE'. The 'S3 - Testing' sprint contains two items: SCRUM-13 and SCRUM-14, both marked as 'TO DO'. The right sidebar displays details for the selected item, SCRUM-102, titled 'Viết mục 5 - User Interface and User Experience Design'. It shows the assignee as 'Quản Phạm Trần Anh' and the parent item as 'SCRUM-102: Viết mục 5'.



24127250 - Phan Quang Tiến - 25%

- Task Evidence:





The image displays two screenshots of the Jira web interface, showing a project named "My Software Team".

Top Screenshot: The main view is the "Backlog" tab. It shows a list of work items grouped by sprint. The "S2 - Design" sprint (1 Jul - 26 Jul) is complete. The "S3 - Testing" sprint (6 Jul - 12 Jul) is in progress. The work items are as follows:

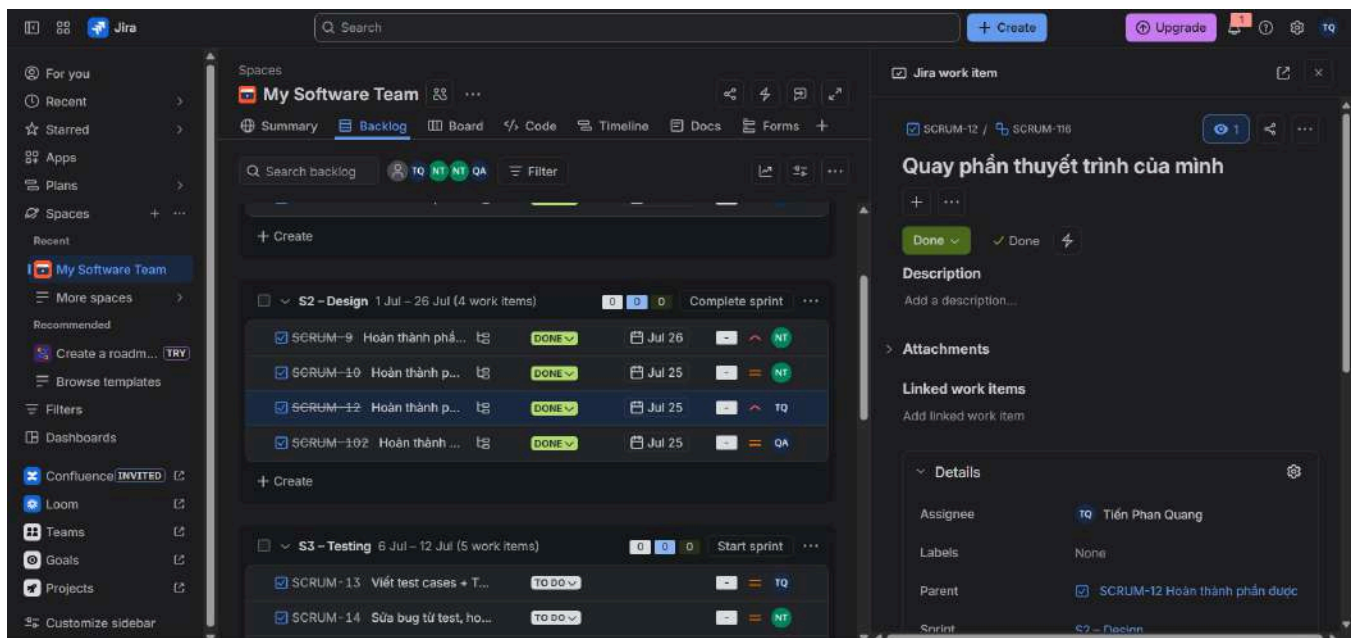
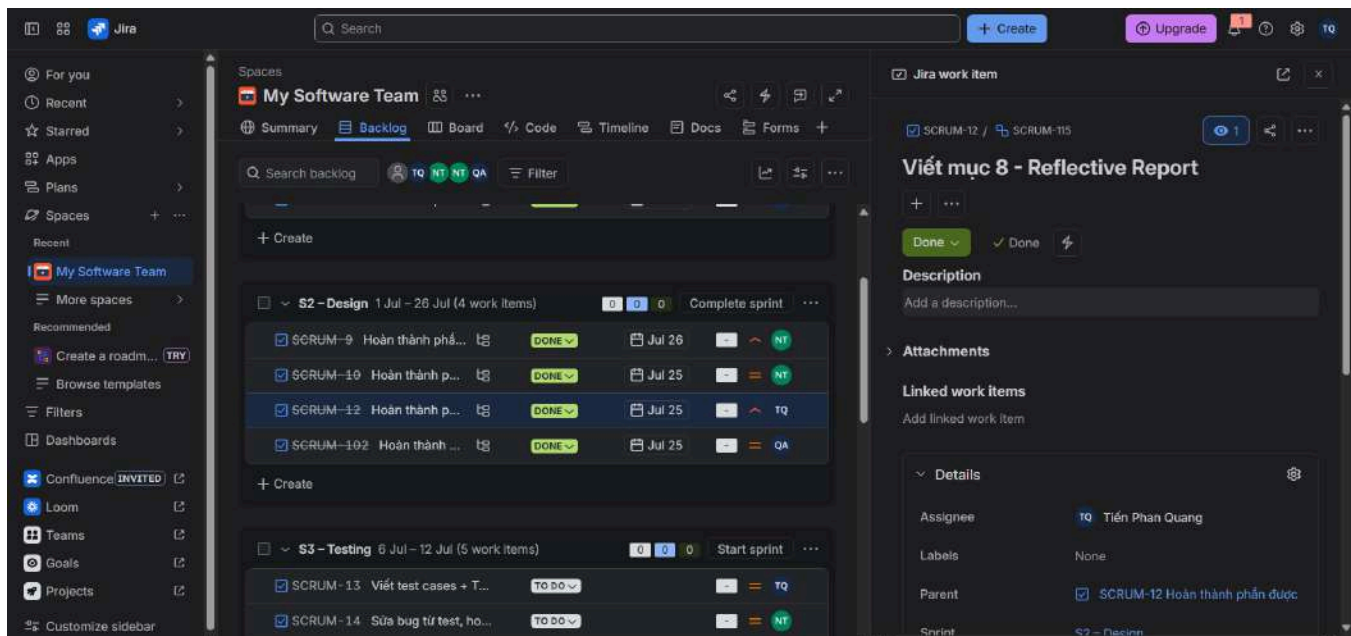
Sprint	Item ID	Summary	Status	Due Date	Assignee
S2 - Design	SCRUM-9	Hoàn thành phần...	DONE	Jul 26	NT
	SCRUM-10	Hoàn thành p...	DONE	Jul 25	NT
	SCRUM-12	Hoàn thành p...	DONE	Jul 25	TQ
	SCRUM-102	Hoàn thành ...	DONE	Jul 25	QA
S3 - Testing	SCRUM-13	Viết test cases + T...	TO DO		TQ
	SCRUM-14	Sửa bug từ test, ho...	TO DO		NT

The right sidebar shows the "Jira work item" detail for "SCRUM-12 / SCRUM-113". The title is "Vẽ phần 1.4 - Data Diagram". The status is "Done". The description is "Add a description...". The assignee is "TQ Tiến Phan Quang". The parent item is "SCRUM-12 Hoàn thành phần được".

Bottom Screenshot: This screenshot is identical to the top one, but the work item detail on the right is for "SCRUM-12 / SCRUM-118" with the title "Viết phần 1.5 - Data Specification".

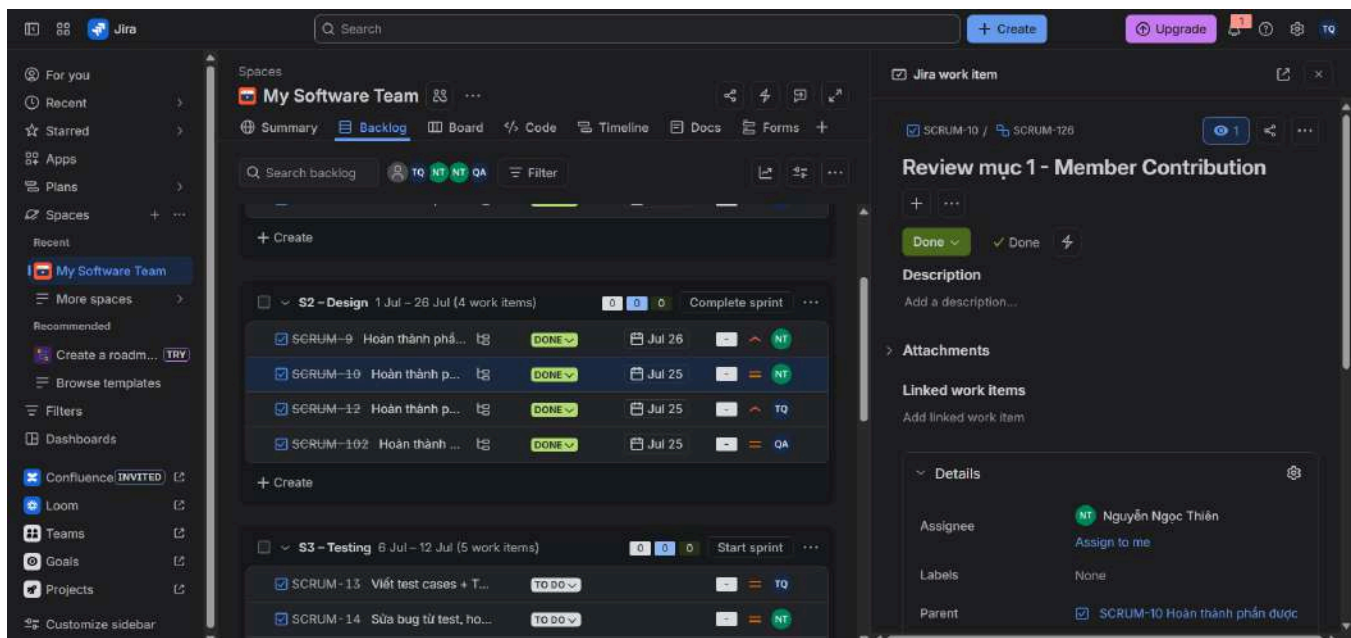
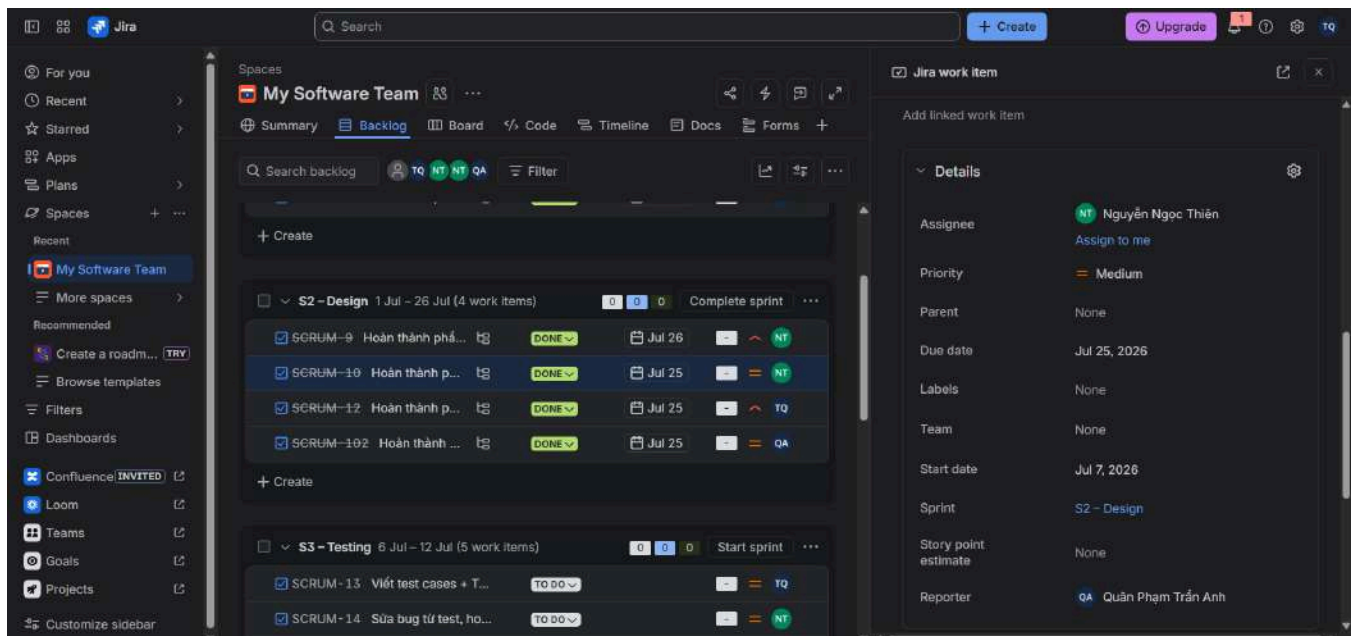
The screenshot shows the Jira interface for 'My Software Team'. The left sidebar contains navigation options like 'For you', 'Recent', 'Starred', 'Apps', 'Plans', 'Spaces', and 'Dashboards'. The main area displays the 'Backlog' for the team, showing two sprints: 'S2 - Design' (1 Jul - 26 Jul) and 'S3 - Testing' (6 Jul - 12 Jul). The 'S2 - Design' sprint is complete, showing four work items: SCRUM-9, SCRUM-10, SCRUM-12, and SCRUM-102, all marked as 'DONE'. The 'S3 - Testing' sprint is in progress, showing two work items: SCRUM-13 and SCRUM-14, both marked as 'TO DO'. The right-hand panel is titled 'Edit phần 1.6 - Screen Diagram' and shows details for work item SCRUM-12, including the assignee 'Tiến Phan Quang' and the parent item 'SCRUM-12 Hoàn thành phần được'.

This screenshot is similar to the one above, showing the Jira interface for 'My Software Team'. The left sidebar is identical. The main area shows the same backlog with 'S2 - Design' and 'S3 - Testing' sprints. The right-hand panel is titled 'Edit phần 1.7 - Screen Specification' and shows details for work item SCRUM-12, including the assignee 'Tiến Phan Quang' and the parent item 'SCRUM-12 Hoàn thành phần được'.



24127545 - Nguyễn Ngọc Thiên - 25%

- Task Evidence:



The screenshot shows the Jira interface for the 'My Software Team' space. The main view is the 'Backlog' tab, displaying a list of work items. The task 'Viết mục 2 - Conceptual Model' (SCRUM-10) is highlighted. The right sidebar shows the details of this task, including the assignee 'Nguyễn Ngọc Thiên' and the parent task 'SCRUM-10 Hoàn thành phần được'.

Sprint	Work Item	Status	Due Date	Assignee
S2 - Design (1 Jul - 26 Jul)	SCRUM-9 Hoàn thành ph...	DONE	Jul 26	NT
	SCRUM-10 Hoàn thành p...	DONE	Jul 25	NT
	SCRUM-12 Hoàn thành p...	DONE	Jul 25	TQ
	SCRUM-102 Hoàn thành ...	DONE	Jul 25	QA
S3 - Testing (6 Jul - 12 Jul)	SCRUM-13 Viết test cases + T...	TO DO		TQ
	SCRUM-14 Sửa bug từ test, ho...	TO DO		NT

The screenshot shows the Jira interface for the 'My Software Team' space. The main view is the 'Backlog' tab, displaying a list of work items. The task 'Edit phần 1.1 - Architecture Diagram' (SCRUM-121) is highlighted. The right sidebar shows the details of this task, including the assignee 'Nguyễn Ngọc Thiên' and the parent task 'SCRUM-10 Hoàn thành phần được'.

Sprint	Work Item	Status	Due Date	Assignee
S2 - Design (1 Jul - 26 Jul)	SCRUM-9 Hoàn thành ph...	DONE	Jul 26	NT
	SCRUM-10 Hoàn thành p...	DONE	Jul 25	NT
	SCRUM-12 Hoàn thành p...	DONE	Jul 25	TQ
	SCRUM-102 Hoàn thành ...	DONE	Jul 25	QA
S3 - Testing (6 Jul - 12 Jul)	SCRUM-13 Viết test cases + T...	TO DO		TQ
	SCRUM-14 Sửa bug từ test, ho...	TO DO		NT

Jira Interface - My Software Team

Backlog

S2 - Design 1 Jul - 26 Jul (4 work items) **Complete sprint**

Item	Status	Due Date	Assignee
SCRUM-9 Hoàn thành ph...	DONE	Jul 26	NT
SCRUM-10 Hoàn thành p...	DONE	Jul 25	NT
SCRUM-12 Hoàn thành p...	DONE	Jul 25	TQ
SCRUM-102 Hoàn thành ...	DONE	Jul 25	QA

S3 - Testing 6 Jul - 12 Jul (5 work items) **Start sprint**

Item	Status	Due Date	Assignee
SCRUM-13 Viết test cases + T...	TO DO		TQ
SCRUM-14 Sửa bug từ test, ho...	TO DO		NT

Jira work item

Edit phần 1.2 - Class Diagram

Description
Add a description...

Attachments
Add linked work item

Details

- Assignee: NT Nguyễn Ngọc Thiên
- Labels: None
- Parent: SCRUM-10 Hoàn thành phần được

Jira Interface - My Software Team

Backlog

S2 - Design 1 Jul - 26 Jul (4 work items) **Complete sprint**

Item	Status	Due Date	Assignee
SCRUM-9 Hoàn thành ph...	DONE	Jul 26	NT
SCRUM-10 Hoàn thành p...	DONE	Jul 25	NT
SCRUM-12 Hoàn thành p...	DONE	Jul 25	TQ
SCRUM-102 Hoàn thành ...	DONE	Jul 25	QA

S3 - Testing 6 Jul - 12 Jul (5 work items) **Start sprint**

Item	Status	Due Date	Assignee
SCRUM-13 Viết test cases + T...	TO DO		TQ
SCRUM-14 Sửa bug từ test, ho...	TO DO		NT

Jira work item

Edit phần 1.3 - Class Specification

Description
Add a description...

Attachments
Add linked work item

Details

- Assignee: NT Nguyễn Ngọc Thiên
- Labels: None
- Parent: SCRUM-10 Hoàn thành phần được

My Software Team

S2 - Design 1 Jul - 26 Jul (4 work items) Complete sprint

- SCRUM-9 Hoàn thành phần... DONE Jul 26 NT
- SCRUM-10 Hoàn thành p... DONE Jul 25 NT
- SCRUM-12 Hoàn thành p... DONE Jul 25 TQ
- SCRUM-102 Hoàn thành ... DONE Jul 25 QA

S3 - Testing 6 Jul - 12 Jul (5 work items) Start sprint

- SCRUM-13 Viết test cases + T... TO DO TQ
- SCRUM-14 Sửa bug từ test, ho... TO DO NT

Jira work item

SCRUM-10 / SCRUM-124

Edit mục 8 - Reflective Report

Done Done

Description
Add a description...

Attachments
Add linked work item

Details

Assignee: NT Nguyễn Ngọc Thiên
Labels: None
Parent: SCRUM-10 Hoàn thành phần được

My Software Team

S2 - Design 1 Jul - 26 Jul (4 work items) Complete sprint

- SCRUM-9 Hoàn thành phần... DONE Jul 26 NT
- SCRUM-10 Hoàn thành p... DONE Jul 25 NT
- SCRUM-12 Hoàn thành p... DONE Jul 25 TQ
- SCRUM-102 Hoàn thành ... DONE Jul 25 QA

S3 - Testing 6 Jul - 12 Jul (5 work items) Start sprint

- SCRUM-13 Viết test cases + T... TO DO TQ
- SCRUM-14 Sửa bug từ test, ho... TO DO NT

Jira work item

SCRUM-10 / SCRUM-125

Quay phần thuyết trình của mình

Done Done

Description
Add a description...

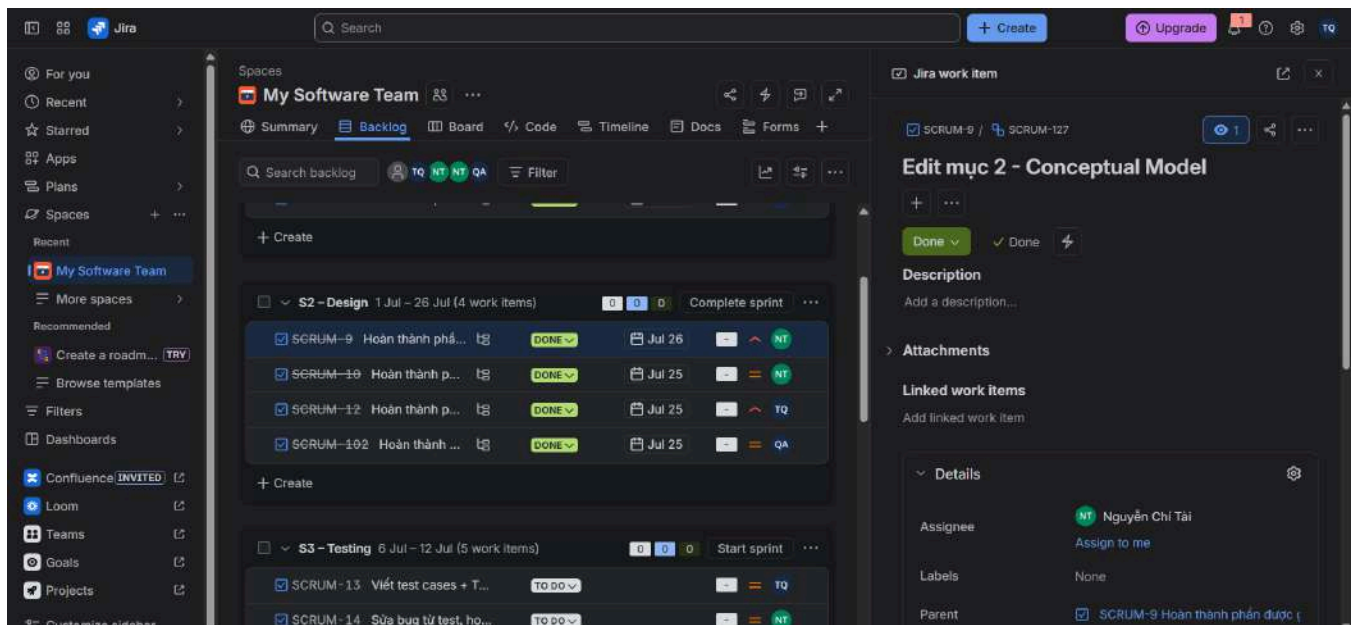
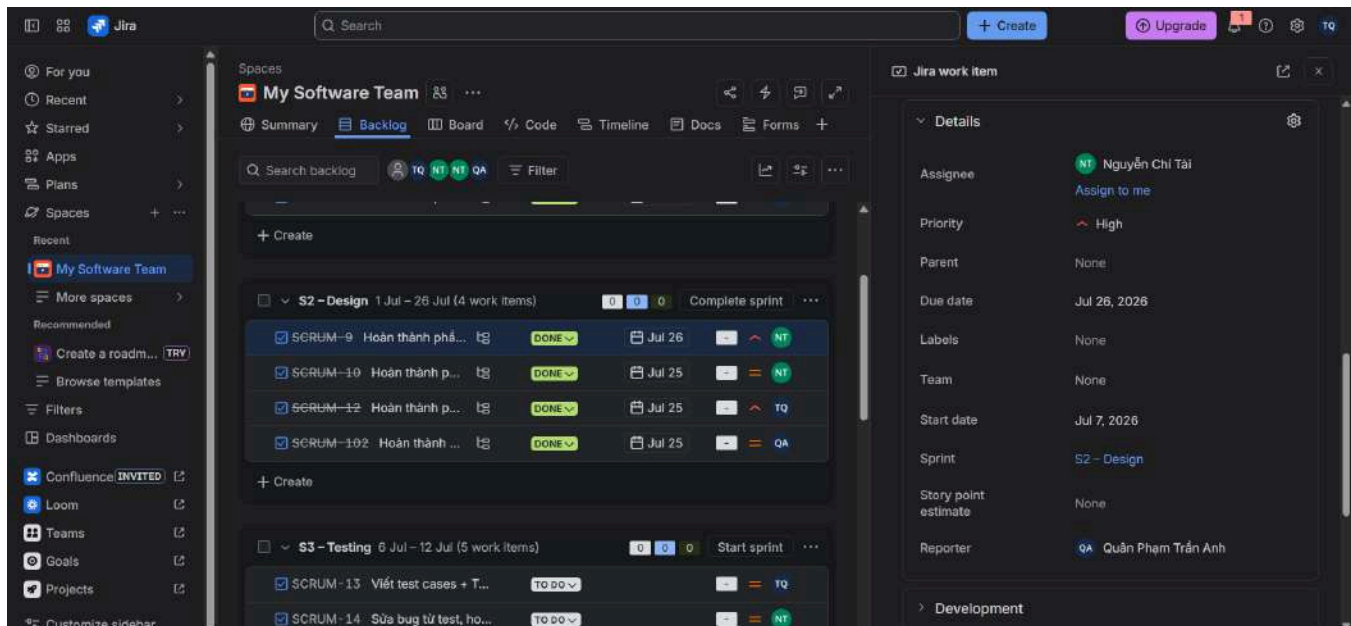
Attachments
Add linked work item

Details

Assignee: NT Nguyễn Ngọc Thiên
Labels: None
Parent: SCRUM-10 Hoàn thành phần được

24127529 - Nguyễn Chí Tài - 25%

- Task Evidence:



The screenshot shows the Jira interface for the 'My Software Team' space. The 'Backlog' tab is selected, displaying a list of work items. The task 'SCRUM-9: Hoàn thành phần 1.1 - Architecture Diagram' is highlighted. The right sidebar shows the details for this task, including the assignee 'Nguyễn Chí Tài' and the parent task 'SCRUM-9: Hoàn thành phần 1.1 - Architecture Diagram'.

Work Item ID	Task Name	Status	Due Date	Assignee
SCRUM-9	Hoàn thành phần 1.1 - Architecture Diagram	DONE	Jul 28	NT
SCRUM-10	Hoàn thành phần 1.2 - Class Diagram	DONE	Jul 25	NT
SCRUM-12	Hoàn thành phần 1.3 - Sequence Diagram	DONE	Jul 25	TQ
SCRUM-102	Hoàn thành phần 1.4 - Use Case Diagram	DONE	Jul 25	QA

The screenshot shows the Jira interface for the 'My Software Team' space. The 'Backlog' tab is selected, displaying a list of work items. The task 'SCRUM-10: Hoàn thành phần 1.2 - Class Diagram' is highlighted. The right sidebar shows the details for this task, including the assignee 'Nguyễn Chí Tài' and the parent task 'SCRUM-9: Hoàn thành phần 1.1 - Architecture Diagram'.

Work Item ID	Task Name	Status	Due Date	Assignee
SCRUM-9	Hoàn thành phần 1.1 - Architecture Diagram	DONE	Jul 28	NT
SCRUM-10	Hoàn thành phần 1.2 - Class Diagram	DONE	Jul 25	NT
SCRUM-12	Hoàn thành phần 1.3 - Sequence Diagram	DONE	Jul 25	TQ
SCRUM-102	Hoàn thành phần 1.4 - Use Case Diagram	DONE	Jul 25	QA

Jira work item

SCRUM-9 / SCRUM-130

Viết phần 1.3 - Class Specification

Description
Add a description...

Attachments
Add attachment

Linked work items
Add linked work item

Details

Assignee: NT Nguyễn Chí Tài
Assign to me

Labels: None

Parent: SCRUM-9 Hoàn thành phần được t

Jira work item

SCRUM-9 / SCRUM-131

Edit phần 1.4 - Data Diagram

Description
Add a description...

Attachments
Add attachment

Linked work items
Add linked work item

Details

Assignee: NT Nguyễn Chí Tài
Assign to me

Labels: None

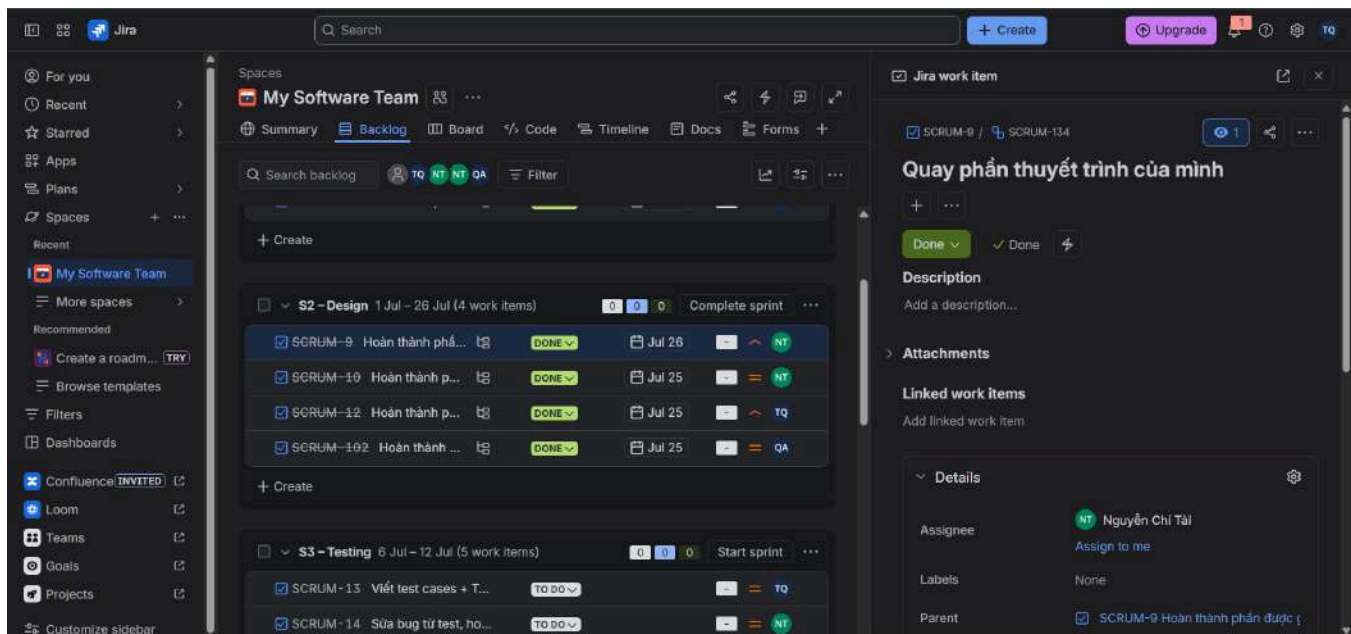
Parent: SCRUM-9 Hoàn thành phần được t

The screenshot shows the Jira interface for the 'My Software Team' space. The main view is the 'Backlog' tab, displaying a list of work items. The right sidebar shows the details of a selected work item, 'Edit phần 1.5 - Data Specification' (SCRUM-9 / SCRUM-132). The work item is assigned to 'Nguyễn Chí Tài' and has a status of 'Done'. The description field is empty, and there are no attachments or linked work items.

Sprint	Work Item	Status	Due Date	Assignee
S2 - Design (1 Jul - 26 Jul)	SCRUM-9: Hoàn thành ph...	DONE	Jul 26	NT
	SCRUM-10: Hoàn thành p...	DONE	Jul 25	NT
	SCRUM-12: Hoàn thành p...	DONE	Jul 25	TQ
	SCRUM-102: Hoàn thành ...	DONE	Jul 25	QA
S3 - Testing (6 Jul - 12 Jul)	SCRUM-13: Viết test cases + T...	TO DO		TQ
	SCRUM-14: Sửa bug từ test, ho...	TO DO		NT

The screenshot shows the Jira interface for the 'My Software Team' space. The main view is the 'Backlog' tab, displaying a list of work items. The right sidebar shows the details of a selected work item, 'Review mục 5 - User Interface and User Experience Design' (SCRUM-9 / SCRUM-133). The work item is assigned to 'Nguyễn Chí Tài' and has a status of 'Done'. The description field is empty, and there are no attachments or linked work items.

Sprint	Work Item	Status	Due Date	Assignee
S2 - Design (1 Jul - 26 Jul)	SCRUM-9: Hoàn thành ph...	DONE	Jul 26	NT
	SCRUM-10: Hoàn thành p...	DONE	Jul 25	NT
	SCRUM-12: Hoàn thành p...	DONE	Jul 25	TQ
	SCRUM-102: Hoàn thành ...	DONE	Jul 25	QA
S3 - Testing (6 Jul - 12 Jul)	SCRUM-13: Viết test cases + T...	TO DO		TQ
	SCRUM-14: Sửa bug từ test, ho...	TO DO		NT



2

Conceptual Model

Please include the ID and full name of the student who authored, reviewed, or updated this section.

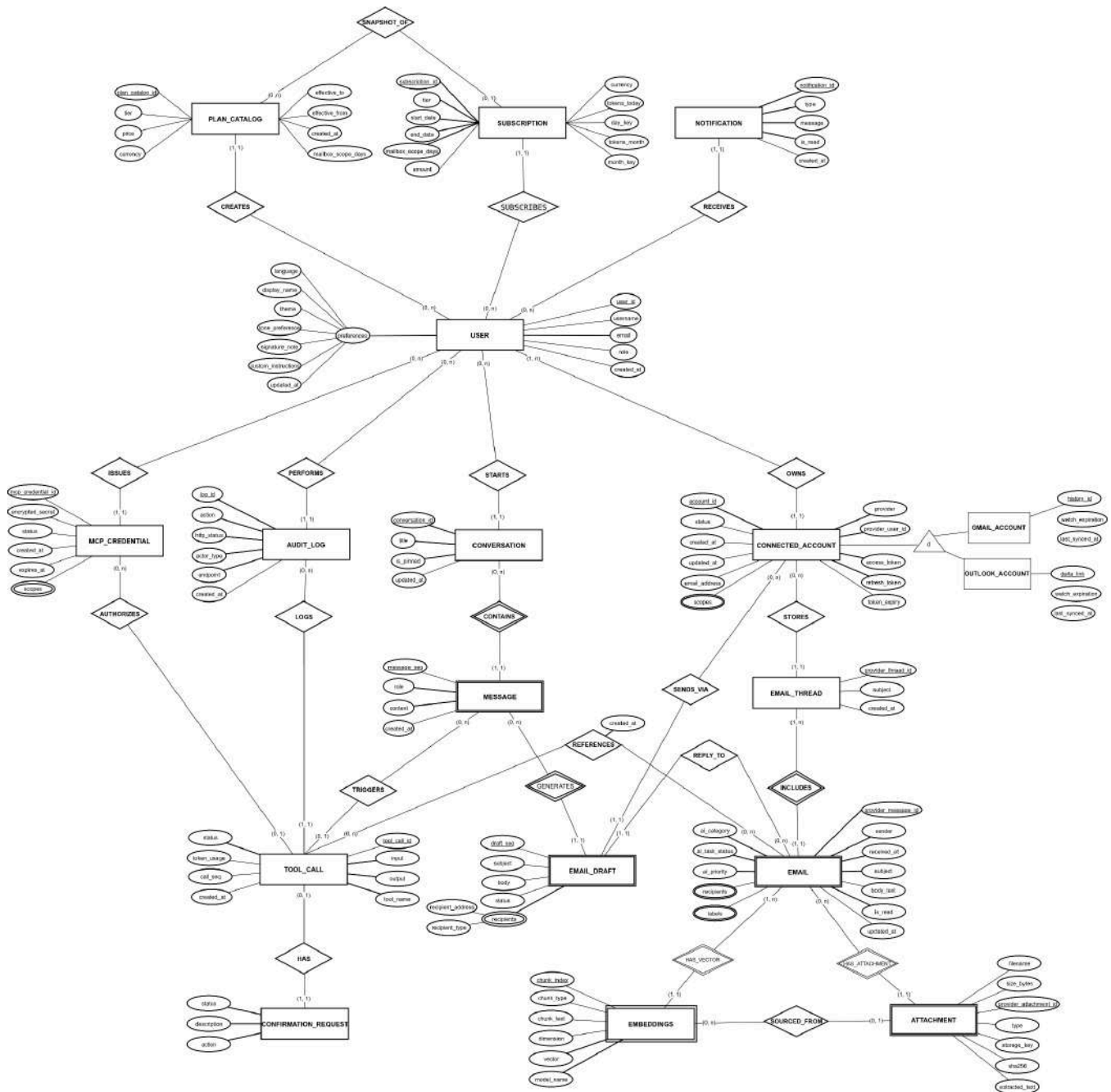
Written by: 24127545 – Nguyễn Ngọc Thiên

Edited by: 24127529 – Nguyễn Chí Tài

Reviewed by: 24127226 – Phạm Trần Anh Quân

[Present a diagram illustrating the semantic entities within the software. This section can be created using an Extended Entity–Relationship (EER) model generated by **MySQL Workbench** or **PowerDesigner**]

<https://drive.google.com/file/d/1uLyotWICfAd-0c09GBthJVRbouPaF_DM/view?usp=sharing>



3 Architectural Design

1.1 Architecture Diagram

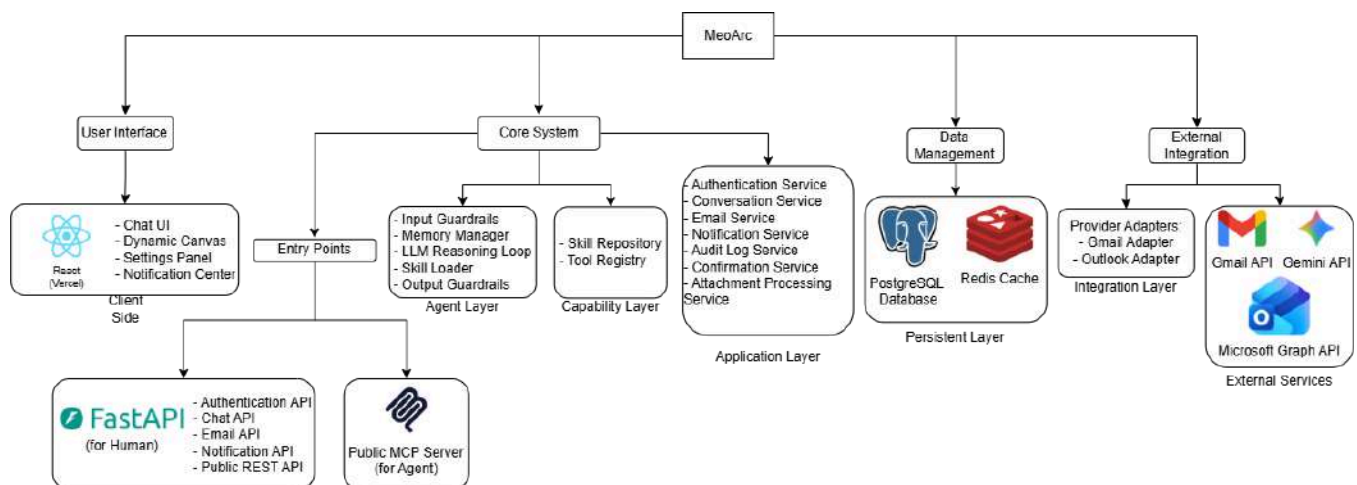
Please include the ID and full name of the student who authored, reviewed, or updated this section.

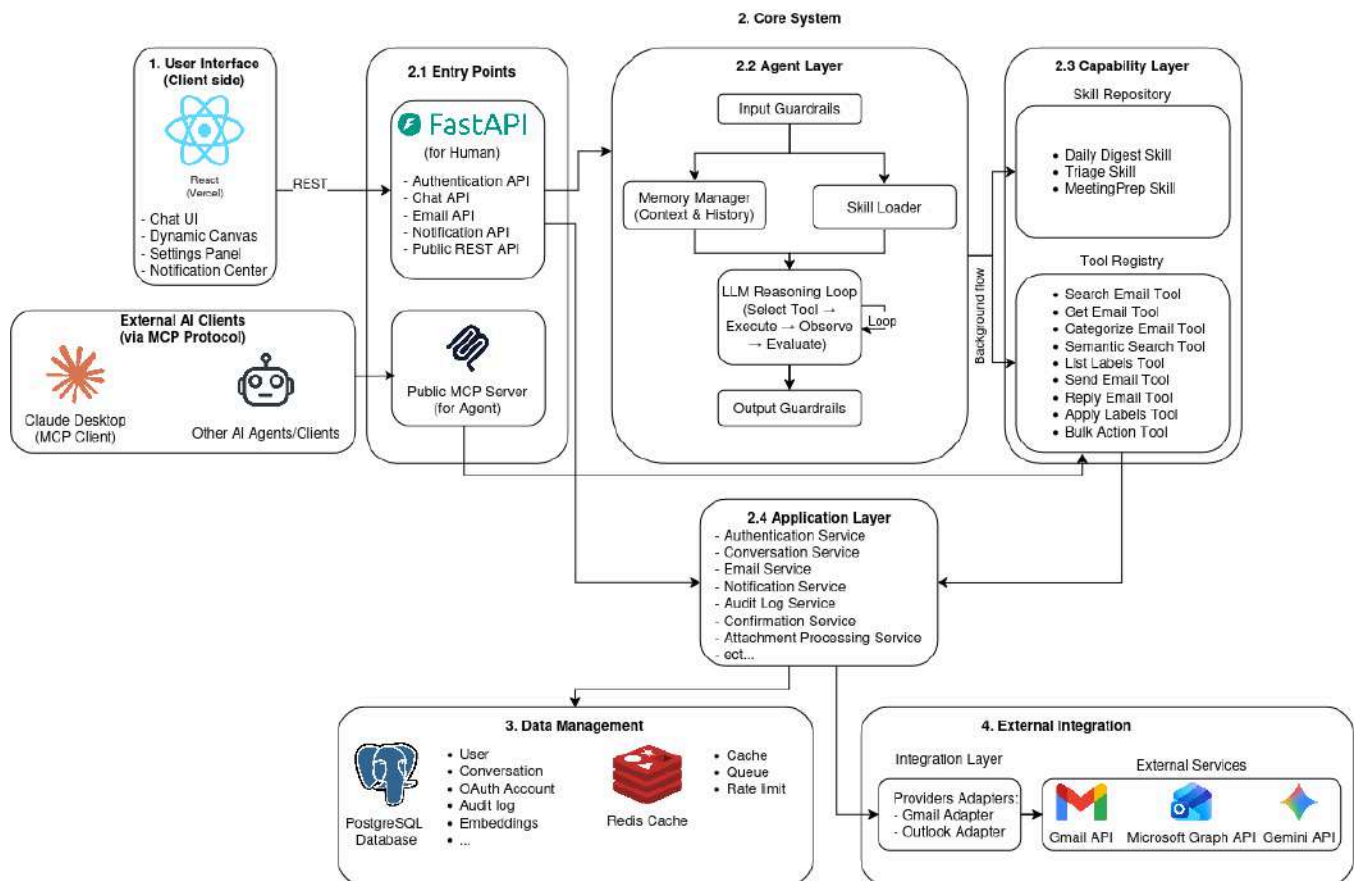
Written by: 24127529 – Nguyễn Chí Tài

Edited by: 24127545 – Nguyễn Ngọc Thiên

Reviewed by: 24127226 – Phạm Trần Anh Quân

https://drive.google.com/file/d/1kYf6xjSLPdOXf98_i_HCPB4vVuSr2SY2/view?usp=drive_link





https://drive.google.com/file/d/1OKoT67BLToAhbLr4Lf59CapPpbpbShKY/view?usp=drive_link

1.2 Class Diagram

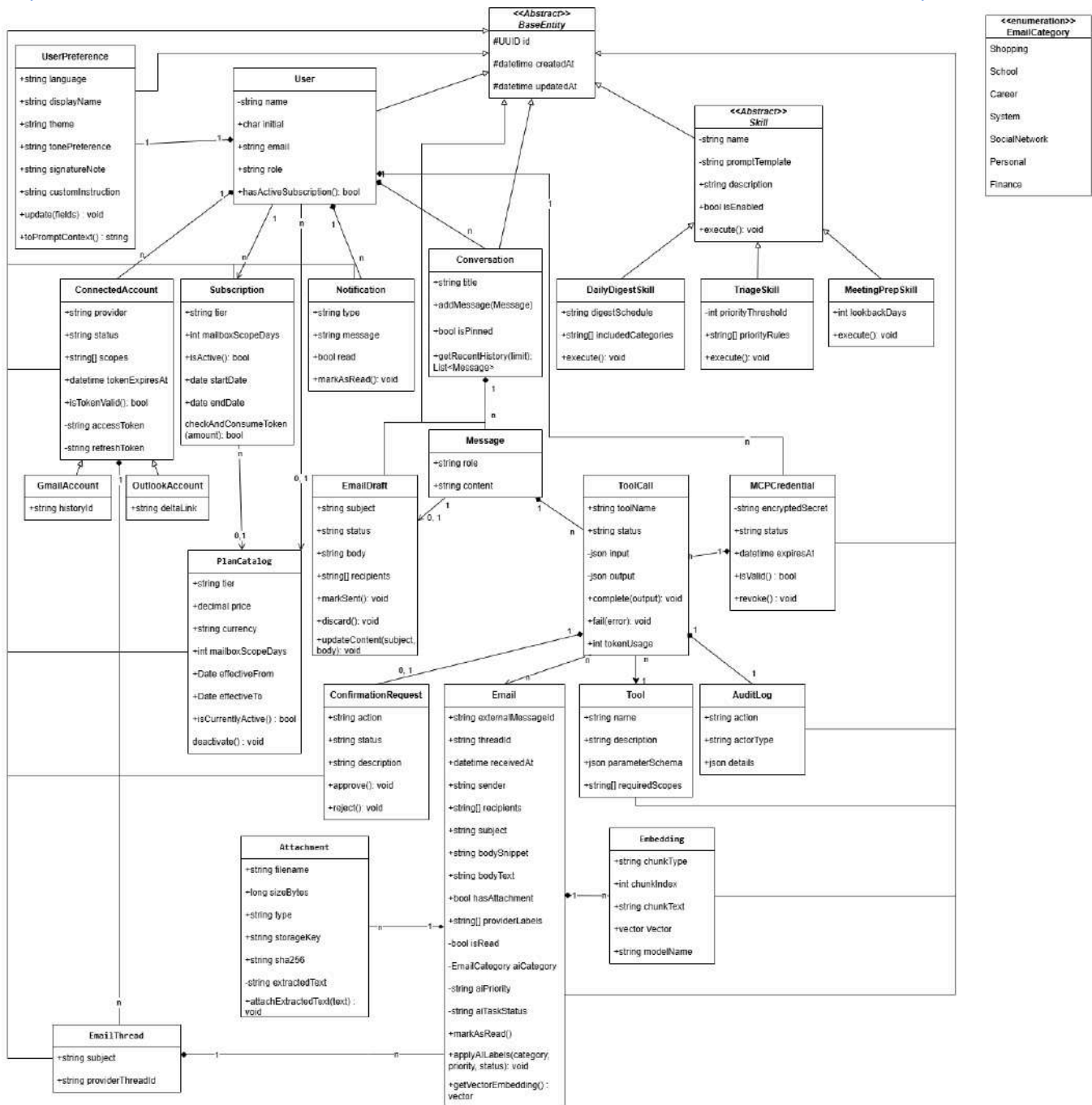
Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by: 24127529 – Nguyễn Chí Tài

Edited by: 24127545 – Nguyễn Ngọc Thiên

Reviewed by: 24127226 – Phạm Trần Anh Quân

https://drive.google.com/file/d/1z9weiAiY75v5bk40R9ev2ZsSOYMCdZ09/view?usp=drive_link



1.3 Class Specifications

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by: 24127529 – Nguyễn Chí Tài

Edited by: 24127545 – Nguyễn Ngọc Thiên

Reviewed by: 24127226 – Phạm Trần Anh Quân

1.3.1 Class “User”

Inherit from: BaseEntity

Attributes are inherited:

- Protected: id, createdAt, updatedAt

Properties:

Seq	Property	Modifier	Constraint	Description
1	name	Private	Not null	The user's display name.
2	initial	Public	1 character	The first letter of name, used for the avatar disc in the UI. This is a display-derived value with no corresponding dedicated column in the USER table of the Data Specification.
3	email	Public	Not null, unique	The primary email used for authentication via Google/Microsoft OAuth.
4	role	Public	Not null, default = "user"	Access role: user or admin.

Operations:

Seq	Operation	Modifier	Constraint	Description
5	hasActiveSubscription() : bool	Public	True only when exactly one Subscription record exists with end_date IS NULL (per the unique index ux_subscription_one_active_per_user)	Checks whether the user currently has an active subscription; the result determines the AI processing scope (mailbox scan window).

1.3.2 Class “Conversation”

Inherit from: BaseEntity

Attributes are inherited:

- Protected: id, createdAt, updatedAt

Properties:

Seq	Property	Modifier	Constraint	Description
1	title	Public	default=Automatically generated after the first message.	Display title of the conversation, automatically generated from the first user message when not explicitly provided.

Operations:

Seq	Operation	Modifier	Constraint	Description
2	addMessage(msg: Message) : void	Public	The message must not belong to any other conversation.	Ensures that every Message is associated with exactly one Conversation, consistent with the composition relationship.
3	getRecentHistory(limit: int) : List<Message>	Public	limit > 0, default=20	Returns the N most recent messages in time – serving the Memory Manager load context (UC007 step 3), avoiding exceeding the token limit when calling LLM.

1.3.3 Class “Message”

Inherit from: BaseEntity

Attributes are inherited:

- Protected: id, createdAt, updatedAt

Properties:

Seq	Property	Modifier	Constraint	Description
1	role	Public	One of {“user”, “assistant”}	Distinguish between user and agent messages – this is mandatory for

				proper formatting when sending to the Gemini API.
2	content	Public	Not null	Message content – read directly by Agent Layer (context) and Chat UI (display).

1.3.4 Class “ToolCall”

Inherit from: BaseEntity

Attributes are inherited:

- Protected: id, createdAt, updatedAt

Properties:

Seq	Property	Modifier	Constraint	Description
1	toolName	Public	Not null	The tool name is saved independently to retain its value even if the original tool is later renamed/deleted.
2	status	Public	One of {"pending", "completed", "failed"}	The Agent Layer reads this field in step "Observe" (UC007) to determine the next step in the reasoning loop.
3	input	Private	Valid JSON	The parameters passed when calling the tool are private because they are only for internal debugging and are not exposed to the UI/MCP.
4	output	Private	Nullable (null when status = "pending")	The result returned after the tool finishes running is written by the complete() method after the tool finishes running.
5	tokenUsage	Public	Non-negative integer, default = 0	Number of tokens consumed by the Gemini API call for this tool invocation, used to track and optimize operating cost.

Operations:

Seq	Operation	Modifier	Constraint	Description
6	complete(output) : void	Public	Only call when status= "pending"	Update status to "completed" and save the output; Do not call any other services, just record the result.
7	fail(error) : void	Public	Only call when status= "pending"	Update status = "failed", save the error to the output – The

				Agent Layer uses this to decide whether to retry or report the error to the user.
--	--	--	--	---

1.3.5 Class “ConfirmationRequest”

Inherit from: BaseEntity

Attributes are inherited:

- Protected: id, createdAt, updatedAt

Description:

Realised in the data design as a one-to-one extension table keyed on `tool_call_id` (§1.5.23), not as a weak entity. A `UNIQUE(tool_call_id)` constraint already limits it to one row per tool call, which leaves a partial key with no second value to distinguish.

Properties:

Seq	Property	Modifier	Constraint	Description
1	action	Public	Not null	The technical name of the action to be confirmed (Ex: "bulk_delete")
2	description	Public	Not null	A clear and easy-to-understand description displayed to the user (Ex: "Xóa 15 email đã chọn?") - serving human-in-the-loop needs.
3	status	Public	one of {"pending", "approved", "rejected"}, default = "pending"	Pending/Processed status.

Operations:

Seq	Operation	Modifier	Constraint	Description
4	approve() : void	Public	Only call when status = "pending"	User agrees - The associated ToolCall will continue executing the action after this call.
5	reject() : void	Public	Only call when status = "pending"	User refused - ToolCall binding stopped, not executed

1.3.6 Class “Tool”

Inherit from: BaseEntity

Attributes are inherited:

- Protected: id, createdAt, updatedAt

Properties:

Seq	Property	Modifier	Constraint	Description
1	name	Public	Not null, unique	Tool name - The Agent Layer reads this to select the tool, and the MCP Server exposes it to the external agent.
2	description	Public	Not null	Function description - used in the function-calling definition sent to the Gemini API
3	parameterSchema	Public	Valid JSON	Define the input parameter structure - required by NFR-COM-02 (MCP must comply with the Model Context Protocol spec)
4	requiredScopes	Public	A subarray of valid logical permission values (e.g. "read", "send", "modify")	Logical permission(s) required for this tool to run, resolved to the correct provider-specific OAuth scope (Gmail or Outlook) at execution time by the Integration Layer (Ex: "Send Email Tool" needs the "send" permission, mapped to gmail.send or Mail.Send depending on the connected account).

This class is not persisted as a database table. The tool registry in the source code is the single source of truth for tool names, descriptions and parameter schemas; storing them again in the database would create two definitions that drift apart, and the one the system actually executes would always be the code's.

1.3.7 Class "Skill"

Inherit from: BaseEntity

Attributes are inherited:

- Protected: id, createdAt, updatedAt

Properties:

Seq	Property	Modifier	Constraint	Description
1	name	Private	Not null, unique	The technical ID is used for the Skill Loader to look up in the Skill Repository (UC007 step 3).

2	description	Public	Not null	Natural language description - content used for semantic search when Skill Loader finds a matching skill.
3	promptTemplate	Private	Not null	The system prompt defines how LLMs behave – it's absolutely not public to avoid being leaked via MCP.
4	isEnabled	Public	Default=true	Allows temporarily disabling a skill without deleting it from the system.

Operations:

Seq	Operation	Modifier	Constraint	Description
5	execute() : void	Public (abstract)	There are no functions in Skill.	The abstract method - each subclass overrides itself according to its own logic - is the anchor point of all skills in the Skill Repository.

1.3.8 Class "TriageSkill"**Inherit from:** Skill**Attributes are inherited:**

- Protected: id, createdAt, updatedAt
- Public: description, isEnabled, execute()

Properties:

Seq	Property	Modifier	Constraint	Description
1	priorityThreshold	Private	Integer (0->100)	The threshold score for deciding whether to assign Email.aiPriority = "High" - a parameter for fine-tuning the internal algorithm.
2	priorityRules	Public	An array of strings, which can be empty.	Additional rules defined by user/admin (Ex: "Email từ giảng viên luôn được ưu tiên (High priority)")

Operations:

Seq	Operation	Modifier	Constraint	Description
3	execute() : void	Public (override)	Inherited from Skill	Read Email.bodyText, call the Gemini Categorize API, call

				Email.applyAllLabels() to write the result — this is where UC009 (Categorize Email) is executed.
--	--	--	--	--

1.3.9 Class “Email”

Inherit from: BaseEntity

Attributes are inherited:

- Protected: id, createdAt, updatedAt

Properties:

Seq	Property	Modifier	Constraint	Description
1	externalMessageId	Public	Not null; must be unique combined with threadId	The original message ID assigned by the provider (Gmail/Outlook).
2	threadId	Public	Not null (FK → EmailThread)	Reference to the conversation thread this message belongs to.
3	receivedAt	Public	Not null	The time the email was received from the provider.
4	sender	Public	Not null	The sender's email address.
5	recipient	Public	—	The recipient's email address.
6	subject	Public	Nullable	The email's subject line.
7	bodySnippet	Public	Nullable	A short preview snippet of the email body.
8	bodyText	Public	Nullable	The full plain-text content of the email.
9	hasAttachment	Public	Default = false	Indicates whether the email has an attachment.
10	providerLabels	Public	Array of strings	The original labels assigned by the provider.
11	isRead	Private	Default = false	Read status; only changed through markAsRead().
12	aiCategory	Private	EmailCategory enum type; always assigned a best-effort value once classified	The classification label determined by the AI Agent (UC009).
13	aiPriority	Private	Values: High, Medium, Low; only assigned when task-like	The priority level determined by the AI.

14	aiTaskStatus	Private	Values: Todo, Waiting, Done; only assigned when task-like	The task state determined by the AI.
----	--------------	---------	---	--------------------------------------

Operations:

Seq	Operation	Modifier	Constraint	Description
15	markAsRead() : void	Public	Idempotent - calling it again does not cause errors	Sets isRead = true, then triggers synchronization back to the source mail provider (Gmail or Outlook) through the Email Service.
16	applyAllLabels(category, priority, status) : void	Public	All three parameters must be set simultaneously	Ensures that the three AI label fields remain consistent after a single Agent analysis.
17	getVectorEmbedding() : vector	Public	Returns the cached value without recomputing	Used by the Semantic Search Tool (FR-03.3) without requiring an additional Gemini API call for each query.

1.3.10 Class “EmailDraft”**Inherit from:** BaseEntity**Attributes are inherited:**

- Protected: id, createdAt, updatedAt

Properties:

Seq	Property	Modifier	Constraint	Description
1	subject, body	Public	Not null when the initial status is not "pending"	Content of the draft email generated by the Agent.
2	recipients	Public	At least one entry with type to	The recipients of the draft, each with an address and a type of to, cc or bcc. Persisted as the EMAIL_DRAFT_RECIPIENT table so that one draft can address several people.

3	status	Public	One of {"pending", "sent", "discarded"}, default = "pending"	Lifecycle status of the draft email.
---	--------	--------	--	--------------------------------------

Operations:

Seq	Operation	Modifier	Constraint	Description
4	updateContent(subject, body) : void	Public	Can only be called when status = "pending"	Updates the draft content by modifying both fields in a single operation.
5	markSent() : void	Public	Can only be called when status = "pending"	Marks the draft as sent without calling the mail provider's send API directly. The actual sending operation is delegated to the Email Service, which routes the request to the correct provider adapter (Gmail or Outlook).
6	discard() : void	Public	Can only be called when status = "pending"	Discards the draft as decided by the user or the Agent.

4

Data Design

1.4 Data Diagram

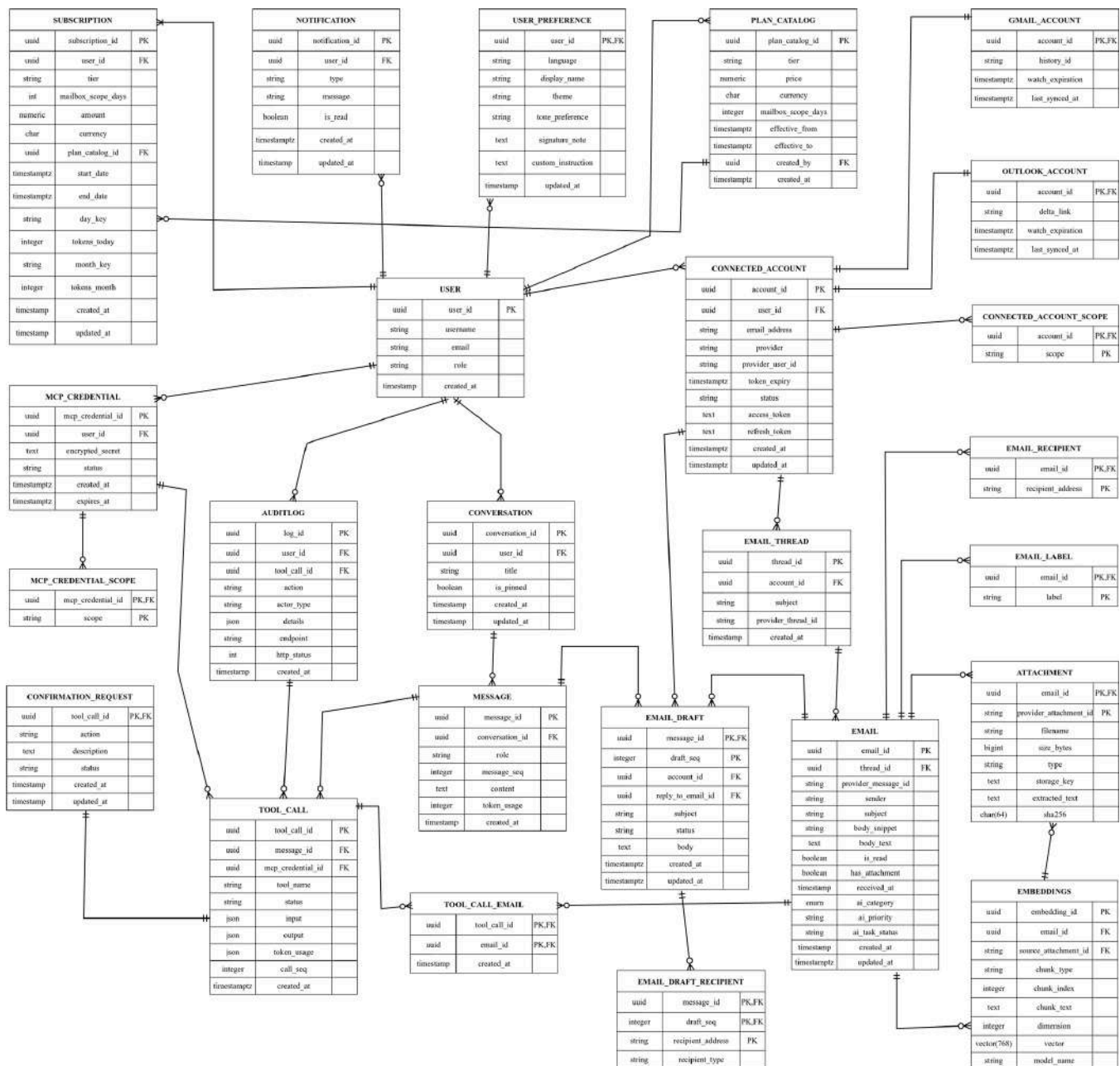
Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by: 24127250 – Phan Quang Tiến

Edited by: 24127529 – Nguyễn Chí Tài

Reviewed by: 24127226 – Phạm Trần Anh Quân

<https://drive.google.com/file/d/1vK7TGPlhCLOufLNGUPRuZK4yLpTaUSK9/view?usp=drive_link>



[Draw the data diagram of the system, identifying the data components that need to be stored and illustrating the relationships between them]

1.5 Data Specification

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by: 24127250 – Phan Quang Tiến

Edited by: 24127529 – Nguyễn Chí Tài

Reviewed by: 24127226 – Phạm Trần Anh Quân

This section describes the detailed structure of the data tables within the PostgreSQL database (including the **pgvector** extension for semantic search functionality). All entities inherit from **BaseEntity**, which provides common fields including a unique identifier (**id**) using the **UUID** format, as well as audit timestamps (**created_at** and **updated_at**)

1.5.1 Table: User

Stores user identification information and personal application preferences. All other entities relate back to this central identity

USER				
Seq	Attribute	Data Type	Key/Constraint	Description
1	user_id	UUID	PK	Unique identifier of the user
2	username	string	NOT NULL	User's display name
3	email	string	UNIQUE, NOT NULL	Primary email address used for identification via Google OAuth.
4	role	string	NOT NULL, DEFAULT user	Access role: user , admin
5	created_at	TIMESTAMP	NOT NULL	Timestamp of record creation

1.5.2 Table: User_Preference

A 1-1 extension of USERS holding personalization settings. Fields like display name, tone, signature, and custom instructions are injected into the AI agent's system prompt at runtime rather than being separate configurations.

USER_PREFERENCE				
Seq	Attribute	Data Type	Key/Constraint	Description
1	user_id	UUID	PK, FK (USERS)	Owner of this preference row (1-1 extension of USERS)
2	language	string	NOT NULL, DEFAULT <code>vi</code>	Preferred UI/response language
3	display_name	string		Preferred display name
4	theme	string	DEFAULT <code>system</code>	UI theme preference
5	tone_preference	string		Preferred writing tone for AI-composed emails
6	signature_note	text		Signature text injected into AI-composed emails
7	custom_instruction	text		Free-form instructions injected into the agent's system prompt
8	updated_at	timestamp	NOT NULL, DEFAULT <code>now()</code>	Last time preferences were changed

1.5.3 Table: Connected_Account

Acts as the parent table for all external email accounts linked to MeoArc. It stores common authentication and status data regardless of the provider.

CONNECTED_ACCOUNT				
Seq	Attribute	Data Type	Key/Constraint	Description
1	account_id	UUID	PK	Unique identifier for the connection record
2	user_id	UUID	FK (USER), NOT NULL	Reference to the owner of the account
3	provider	string	NOT NULL	Service provider (google , microsoft) – discriminator for the subtypes
4	provider_user_id	string	NOT NULL, UNIQUE (provider, provider_user_id)	The unique user ID issued by the external service. The pair with provider prevents the same mailbox being connected twice, which would make every sync run twice
5	email_address	string	NOT NULL	Human-readable address of the connected mailbox; used for display, the From address, and routing push notifications to the right account
6	status	string	NOT NULL	active , revoked , error . Revoking changes the status instead of deleting the row, so existing audit records keep something to point at
7	access_token	text	Encrypted	Short-lived token for calling the provider API
8	refresh_token	text	Encrypted, NULL	Long-lived token used to obtain a new access token. Nullable

				because Google and Microsoft do not always return one on re-consent; NOT NULL would force storing an empty string and lose the distinction between "never had one" and "empty"
9	token_expiry	TIMESTAMPTZ	NULL	When <code>access_token</code> expires. Without it the system cannot know when to refresh and must call, take a 401, and only then refresh
10	created_at	TIMESTAMPTZ	NOT NULL	When the account was first connected
11	updated_at	TIMESTAMPTZ	NOT NULL	Last token refresh or status change

1.5.4 Table: Connected_Account_Scope

CONNECTED_ACCOUNT_SCOPE				
Seq	Attribute	Data Type	Key/Constraint	Description
1	account_id	UUID	FK (CONNECTED_ACCOUNT)	Connected account this scope belongs to
2	scope	string	NOT NULL	OAuth scope string granted

PK(account_id, scope)

1.5.5 Table: Gmail_Account

A specialized child table that inherits from **CONNECTED_ACCOUNT**, containing metadata specific to the Google Gmail API.

GMAIL_ACCOUNT				
Seq	Attribute	Data Type	Key/Constraint	Description

1	account_id	UUID	PK, FK (CONNECTED_ACCOUNT)	Inherits the parent connection
2	history_id	string	NULL	Gmail change cursor for incremental sync
3	watch_expiration	TIMESTAMPTZ	NULL	Gmail push subscriptions expire after about 7 days. Without this the renewal deadline is unknown, and one day mail silently stops arriving with no error anywhere
4	last_synced_at	TIMESTAMPTZ	NULL	Last successful sync, also shown in the UI

1.5.6 Table: Outlook_Account

A specialized child table that inherits from **CONNECTED_ACCOUNT**, specifically designed to handle metadata required for the Microsoft Graph API (Outlook).

OUTLOOK_ACCOUNT				
Seq	Attribute	Data Type	Key/Constraint	Description
1	account_id	UUID	PK, FK (CONNECTED_ACCOUNT)	Inherits the parent connection
2	delta_link	text	NULL	Microsoft Graph delta query cursor
3	watch_expiration	TIMESTAMPTZ	NULL	Outlook push subscriptions expire after about 7 days. Without this the renewal deadline is unknown, and one day mail silently stops arriving with no error anywhere

4	last_synced_at	TIMESTAMPTZ	NULL	Last successful sync
---	----------------	-------------	------	----------------------

1.5.7 Table: Subscription

Manages user service tiers and associated AI processing limitations, such as the historical time window for mailbox scanning.

SUBSCRIPTION				
Seq	Attribute	Data Type	Key/Constraint	Description
1	subscription_id	UUID	PK	Unique identifier for the subscription record
2	user_id	UUID	FK(USER), NOT NULL	Reference to the user owning the subscription
3	tier	string	NOT NULL	Service level (e.g. 'Free', 'Pro', 'Pro Max')
4	mailbox_scope_days	integer	NOT NULL	Maximum days of mailbox history the AI may scan (NFR-SCO-01). Snapshotted from plan_catalog when the period starts, not read back from it: otherwise editing the price list would silently change the scope of every existing user, including ones who paid for the old value
5	amount	NUMERIC(12,2)	NOT NULL	Amount actually charged for this period, snapshotted at transaction time rather than joined from the current price list
6	currency	CHAR(3)	NOT NULL, DEFAULT VND	ISO 4217 code

7	plan_catalog_id	UUID	FK (PLAN_CATALOG), NULL	Traceability only – never used to recompute revenue
8	start_date	TIMESTAMPTZ	NOT NULL	When this period began
9	end_date	TIMESTAMPTZ	NULL	NULL means this period is current
10	day_key	string	NOT NULL	Current day bucket (YYYY-MM-DD) for the token counter
11	tokens_today	integer	NOT NULL, DEFAULT 0	Tokens consumed today
12	month_key	string	NOT NULL	Current month bucket (YYYY-MM)
13	tokens_month	integer	NOT NULL, DEFAULT 0	Tokens consumed this month
14	created_at	TIMESTAMPTZ	NOT NULL	Record creation
15	updated_at	TIMESTAMPTZ	NOT NULL	Updated on renewal or tier change

Constraint: `CREATE UNIQUE INDEX ux_subscription_one_active_per_user ON subscription(user_id) WHERE end_date IS NULL` – prevents two concurrent active periods for one user, which a retried payment webhook could otherwise create without any error being raised.

The token counters live here rather than being summed from `tool_call` at read time. Quota must be checked before every chat turn, and a `SUM` across `tool_call` → `message` → `conversation` → `user` on the fastest-growing table in the system would not hold up.

1.5.8 Table: Notification

Stores system alerts and task completion summaries for the user, supporting the notification center features.

NOTIFICATION				
Seq	Attribute	Data Type	Key/Constraint	Description
1	notification_id	UUID	PK	Unique identifier for the notification
2	user_id	UUID	FK(USER)	Reference to the user receiving the notification
3	type	string	NOT NULL	The notification category (e.g. 'Info', 'Error', 'TaskSummary')
4	message	string	NOT NULL	The text content of the alert or notification
5	is_read	Boolean	Default: False	Indicates if the notification has been seen by the user
6	created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	When the notification was raised. Without it notifications cannot be ordered by time, which is essentially the only thing done with this table
7	updated_at	Timestamp	NOT NULL	Updated when the notification status changes

1.5.9 Table: Email_Thread

Groups related emails into conversations as defined by the service provider (Gmail/Outlook), enabling threaded views and context-aware AI reasoning

EMAIL_THREAD				
Seq	Attribute	Data Type	Key/Constraint	Description
1	thread_id	UUID	PK	Unique identifier for the thread record

2	account_id	UUID	FK(CONNECTED_ACCOUNT), NOT NULL	Reference to the account this thread belongs to
3	subject	string		The common subject line for the thread
4	provider_thread_id	string	NOT NULL	The original threadID from the provider (Gmail/Outlook)
5	created_at	timestamp	NOT NULL	Timestamp when the thread was first synchronized

1.5.10 Table: Email

Stores the detailed metadata and content of individual messages. This table serves as the primary source for AI categorization, summarization, and task extraction

EMAIL				
Seq	Attribute	Data Type	Key/Constraint	Description
1	email_id	UUID	PK	Unique identifier for the email (surrogate)
2	thread_id	UUID	FK(EMAILTHREAD), NOT NULL	References to the parent conversation thread
3	provider_message_id	string	NOT NULL	The original message ID from the provider
4	sender	string	NOT NULL	Sender's email address
5	subject	string		Email subject line
6	body_snippet	string		Short preview snippet of the email body
7	body_text	text		Full plain text content of the email

8	is_read	boolean	Default: False	Read/unread status synchronized with the provider
9	ai_category	Enum		AI Classification: Spam, School, Career, Finance, Personal → School, Work, Finance, Social, Shopping, System, Personal
10	ai_priority	string		AI-determined priority: High, Medium, Low
11	ai_task_status	string		AI-determined task state: Todo, Waiting, Done
12	received_at	TIMESTAMP	NOT NULL	The time the email was originally received by the provider
13	created_at	TIMESTAMP	NOT NULL	System synchronization timestamp
14	updated_at	TIMESTAMP TZ	NOT NULL	Updated whenever the AI labels or the read status change
15	has_attachment	boolean	Default: False	Indicate that the email whether it has attachment or not

Identity is **UNIQUE (thread_id, provider_message_id)**. Display order within a thread is **ORDER BY received_at ASC, provider_message_id ASC** – deterministic on its own, because **provider_message_id** is already unique within the thread. An extra sequence column was considered and dropped: assigning gap-free numbers would require locking the parent thread row on every insert, which serialises exactly the hottest path during the initial mailbox sync.

1.5.11 Table: Email_Recipient

EMAIL_RECIPIENT				
Seq	Attribute	Data Type	Key/Constraint	Description

1	email_id	UUID	PK(email_id, recipient_address), FK (EMAIL)	Email this recipient belongs to
2	recipient_address	string	PK (composite)	Recipient email address

1.5.12 Table Email_Label

Multivalued child of **email** — provider labels applied to a message.

EMAIL_LABEL				
Seq	Attribute	Data Type	Key/Constraint	Description
1	email_id	UUID	PK(email_id, label), FK (EMAIL)	Email this recipient belongs to
2	label	string	PK(email_id, label)	Label/tag value

1.5.13 Table: Attachment

Manages metadata and extracted content from email attachments, supporting advanced search and context retrieval

ATTACHMENT				
Seq	Attribute	Data Type	Key/Constraint	Description
1	email_id	UUID	PK (composite), FK (EMAIL)	The email this file is attached to
2	provider_attachment_id	string	PK (composite)	The ID assigned to the file by the provider. Together with email_id this is the real identity, so no surrogate key is needed
3	filename	string	NOT NULL	Name of the attached file

4	size_bytes	BIGINT	NULL	File size
5	type	string	NULL	MIME type
6	storage_key	text	NOT NULL	Logical address used to fetch the file back
7	sha256	CHAR(64)	NOT NULL	Content hash, used to avoid storing the same file twice
8	extracted_text	text	NULL	Text extracted from the document, used as grounding for embeddings and search

1.5.14 Table: Audit_Log

Stores comprehensive system logs for maintenance, security monitoring, and debugging. It tracks both explicit user actions and automated AI agent operations

AUDIT_LOG				
Seq	Attribute	Data Type	Key/Constraint	Description
1	log_id	UUID	PK	Unique identifier for the log entry
2	user_id	UUID	FK(USER), NOT NULL	The user who initiated or is associated with the action
3	tool_call_id	UUID	FK(TOOL_CALL), NOT NULL, UNIQUE	Reference to the specific agent tool execution if applicable
4	action	string	NOT NULL	Description of the action (e.g. 'LOGIN', 'SEND_EMAIL')
5	actor_type	string		The type of actor: 'USER' or 'AI_AGENT'
6	endpoint	string		The specific API endpoint accessed during the action

7	http_status	integer		The HTTP response status code (e.g. 200, 404, 500)
8	details	JSON		Additional contextual metadata about the event
9	created_at	timestamp	NOT NULL, DEFAULT now()	The extract time the log record was created

Index: (**user_id**, **created_at** DESC). The real query is always "this user's activity, most recent first". Two separate indexes on **user_id** and **created_at** cannot serve that as well as one composite index: the database must filter by user and then sort again. This table only ever grows, so the gap widens over time.

1.5.15 Table: Embeddings

Stores high-dimensional numerical vectors and text chunks generated from emails. This enables advanced **Semantic Search** and provides the grounding data for the AI's Retrieval-Augmented Generation (RAG) loop

EMBEDDINGS				
Seq	Attribute	Data Type	Key/Constraint	Description
1	embedding_id	UUID	PK	Unique identifier for the embedding record
2	email_id	UUID	FK (EMAIL), NOT NULL	The email this vector was derived from
3	source_attachment_id	string	NULL, FK (email_id, source_attachment_id) → ATTACHMENT	NULL means the chunk came from the email body; a value means it came from that attachment's extracted text
4	chunk_type	string	NOT NULL	body , header , or attachment — which part of the source this chunk came from

5	chunk_index	integer	NOT NULL, UNIQUE (email_id, chunk_index)	Position of the chunk within its source, and the ordering key for retrieval
6	chunk_text	text	NULL	Raw text this vector was computed from
7	vector	vector(768)	NOT NULL	The numerical array, stored via the pgvector extension
8	dimension	integer	NOT NULL	Size of the vector. Informational: the MVP fixes one model, so the column type carries a literal dimension – pgvector requires a constant at CREATE TABLE time, and an unconstrained vector cannot carry an HNSW index
9	model_name	string	NOT NULL	The embedding model used

Index: **CREATE INDEX ON embedding USING hsw (vector vector_cosine_ops).**
Without it every semantic search is a sequential scan of the whole table.

1.5.16 Table: Tool_Call_Email (Intermediate Table)

Manages the many-to-many (n:m) relationship between **TOOLCALL** and **EMAIL**. It is crucial for auditing because it links a single AI action (like "Archive all receipts") to the specific emails it targeted

TOOL_CALL_EMAIL				
Seq	Attribute	Data Type	Key/Constraint	Description
1	tool_call_id	UUID	PK, FK(TOOL_CALL)	Reference to the AI tool invocation
2	email_id	UUID	PK, FK(EMAIL)	Reference to the email accessed or modified by the tool

3	created_at	timestamp	NOT NULL	Timestamp when the specific association was recorded
---	------------	-----------	----------	--

1.5.17 Table: Conversation

Stores metadata for AI chat sessions, supporting chat history management and organization features such as pinning and custom titles

CONVERSATION				
Seq	Attribute	Data Type	Key/Constraint	Description
1	conversation_id	UUID	PK	Unique identifier for the conversation session
2	user_id	UUID	FK(USER)	Reference to the owner of conversation
3	title	string		Display title (auto-generated by AI or renamed by user)
4	is_pinned	Boolean	Default: False	Indicates if the session is pinned to the top of the list
5	created_at	Timestamp	NOT NULL	Creation timestamp of the session
6	updated_at	Timestamp	NOT NULL	Timestamp of the last message or title update

1.5.18 Table: Message

Stores individual exchanges within a conversation. This table provides the core historical context required for the LLM reasoning loop

MESSAGE

Seq	Attribute	Data Type	Key/Constraint	Description
1	message_id	UUID	PK	Unique identifier for the message
2	conversation_id	UUID	FK(CONVERSATION)	The session this message belongs to
3	role	Enum	NOT NULL	The sender's role: 'user' or 'assistant'
4	content	Text	NOT NULL	The actual text content of the message
5	message_seq	integer		Sequential order of the message within the conversation
6	token_usage	JSON		Records the total tokens consumed by this specific turn
7	created_at	Timestamp	NOT NULL	Timestamp when the message was sent/received

1.5.19 Table: Tool_Call

Records specific instances of tool execution triggered by the AI Agent. It links model intents to actual mailbox operations and tracks execution success or failure

TOOL_CALL				
Seq	Attribute	Data Type	Key/Constraint	Description
1	tool_call_id	UUID	PK	Unique identifier for the tool call
2	message_id	UUID	FK (MESSAGE), NULL	The assistant message that triggered this call – the in-app chat path
3	mcp_credential_id	UUID	FK (MCP_CREDENTIAL), NULL	The external agent credential that triggered it – the MCP path

4	tool_name	string	NOT NULL	Technical name of the tool, e.g. <code>send_email</code>
5	status	string	NOT NULL	<code>pending</code> , <code>running</code> , <code>awaiting_confirmation</code> , <code>success</code> , <code>failed</code>
6	input	JSONB	NOT NULL	Parameters produced by the model
7	output	JSONB	NULL	Raw result returned by the tool
8	token_usage	integer	NULL	Tokens consumed by this call
9	call_seq	integer	NULL, UNIQUE (message_id, call_seq) WHERE message_id IS NOT NULL	Order within one message's set of calls. NULL on the MCP path, which has no parent message to order against
10	created_at	TIMESTAMP TZ	NOT NULL	Record creation

Constraint: `CHECK ((message_id IS NOT NULL)::int + (mcp_credential_id IS NOT NULL)::int = 1)` – a tool call comes from exactly one of the two entry points, never both and never neither.

Index: `(message_id)` and `(mcp_credential_id, created_at DESC)`.

1.5.20 Table: Email_Draft

Stores email content generated by the AI Agent that is awaiting final confirmation, editing, or transmission. It maintains links to the generating message and the target account to ensure context is never lost

EMAIL_DRAFT				
Seq	Attribute	Data Type	Key/Constraint	Description

1	message_id	UUID	PK (composite), FK (MESSAGE)	The assistant message that produced this draft
2	draft_seq	integer	PK (composite)	Order of drafts produced within that message
3	account_id	UUID	FK (CONNECTED_ACCOUNT), NOT NULL	The mailbox the draft will be sent from – required once a user can connect more than one
4	reply_to_email_id	UUID	FK (EMAIL), NULL	The original email when this draft is a reply
5	subject	string	NULL	Proposed subject line
6	body	text	NULL	Proposed body
7	status	string	NOT NULL	pending, sent, discarded
8	created_at	TIMESTAMP TZ	NOT NULL	When the draft was generated
9	updated_at	TIMESTAMP TZ	NOT NULL	Last edit by the user (UC010)

1.5.21 Table: Email_Draft_Recipient

Multivalued child of **email_draft**, replacing the single **recipientAddress** column so that a draft can address more than one person and distinguish To from Cc and Bcc.

EMAIL_DRAFT_RECIPIENT				
Seq	Attribute	Data Type	Key/Constraint	Description
1	message_id	UUID	PK (composite), FK (EMAIL_DRAFT)	Part of the parent draft's key

EMAIL_DRAFT_RECIPIENT				
Seq	Attribute	Data Type	Key/Constraint	Description
2	draft_seq	integer	PK (composite), FK (EMAIL_DRAFT)	Part of the parent draft's key
3	recipient_address	string	PK (composite)	One recipient address
4	recipient_type	string	NOT NULL	to, cc, or bcc

1.5.22 Table: Plan_Catalog

The published price list and policy. Separate from **subscription** so that changing a price never rewrites what past periods were charged.

PLAN_CATALOG				
Seq	Attribute	Data Type	Key/Constraint	Description
1	plan_catalog_id	UUID	PK	Unique identifier for the price entry
2	tier	string	NOT NULL	free, pro, pro_max
3	price	NUMERIC(12,2)	NOT NULL	Listed price
4	currency	CHAR(3)	NOT NULL, DEFAULT VND	ISO 4217 code
5	mailbox_scope_ days	integer	NOT NULL	AI processing window offered by this tier
6	effective_from	TIMESTAMPZ	NOT NULL	When this price takes effect

PLAN_CATALOG				
Seq	Attribute	Data Type	Key/Constraint	Description
7	effective_to	TIMESTAMPTZ	NULL	NULL means currently in effect
8	created_by	UUID	FK (USER), NULL	Administrator who created the entry. Nullable so that the rows seeded at install time, before any administrator exists, can still be inserted
9	created_at	TIMESTAMPTZ	NOT NULL	Record creation

Constraint: **UNIQUE INDEX** `ux_plan_catalog_one_active_per_tier` ON `plan_catalog(tier)` WHERE `effective_to` IS NULL.

1.5.23 Table: Confirmation_Request

Handles the "Human-in-the-loop" approval workflows for risky or irreversible actions proposed by the AI Agent, such as bulk deletions or sending sensitive emails

CONFIRMATION_REQUEST				
Seq	Attribute	Data Type	Key/Constraint	Description
1	tool_call_id	UUID	PK, FK (TOOL_CALL)	The tool execution awaiting approval. Being both the primary and the foreign key enforces at most one confirmation per tool call structurally
2	action	string	NOT NULL	Technical name of the action, e.g. <code>bulk_delete</code>

3	description	text	NOT NULL	Plain-language summary shown to the user, e.g. "Xóa 15 email đã chọn?" – the user approves what they can read, not a function name
4	status	string	NOT NULL, DEFAULT pending	pending, approved, rejected
5	created_at	TIMESTAMPTZ	NOT NULL	When the request was raised
6	updated_at	TIMESTAMPTZ	NOT NULL	Updated when the user responds

`approve()` and `reject()` may only be called while `status = 'pending'`. This single rule is what prevents a double submission from executing an irreversible action twice: once approved the row is no longer pending, so a second click cannot run the action again.

1.5.24 Table MCP_Credential

Credentials issued so that an external agent (for example Claude Desktop) can reach the MeoArc tools without going through the web interface (UC012).

MCP_CREDENTIAL				
Seq	Attribute	Data Type	Key/Constraint	Description
1	mcp_credential_id	UUID	PK	Unique identifier for the credential
2	user_id	UUID	FK (USER), NOT NULL	The user on whose behalf the external agent acts
3	encrypted_secret	text	NOT NULL, UNIQUE	The credential itself, stored encrypted
4	status	string	NOT NULL	active or revoked
5	created_at	TIMESTAMP TZ	NOT NULL	When issued
6	expires_at	TIMESTAMP TZ	NULL	Optional expiry

1.5.25 Table MCP_Credential_Scope

Multivalued child of `mcp_credential` – the permissions this credential carries. An external agent may never exceed them (FR-05.2).

MCP_CREDENTIAL_SCOPE				
Seq	Attribute	Data Type	Key/Constraint	Description
1	mcp_credential_id	UUID	PK(mcp_credential_id, scope), FK (MCP_CREDENTIAL)	The credential
2	scope	string	PK(mcp_credential_id, scope)	One permission

[If using a Database Management System (DBMS), describe the data tables, the information for each data column including attribute name, data type and value constraints, key constraints..., and explain the attribute]

[If using XML/JSON or a self-defined structured file to store data, the file structure, attribute information, data type, and value constraints need to be specifically described. An example of the content for the information storage file should be included]

5

User Interface and User Experience Design

1.6 Screen Diagram

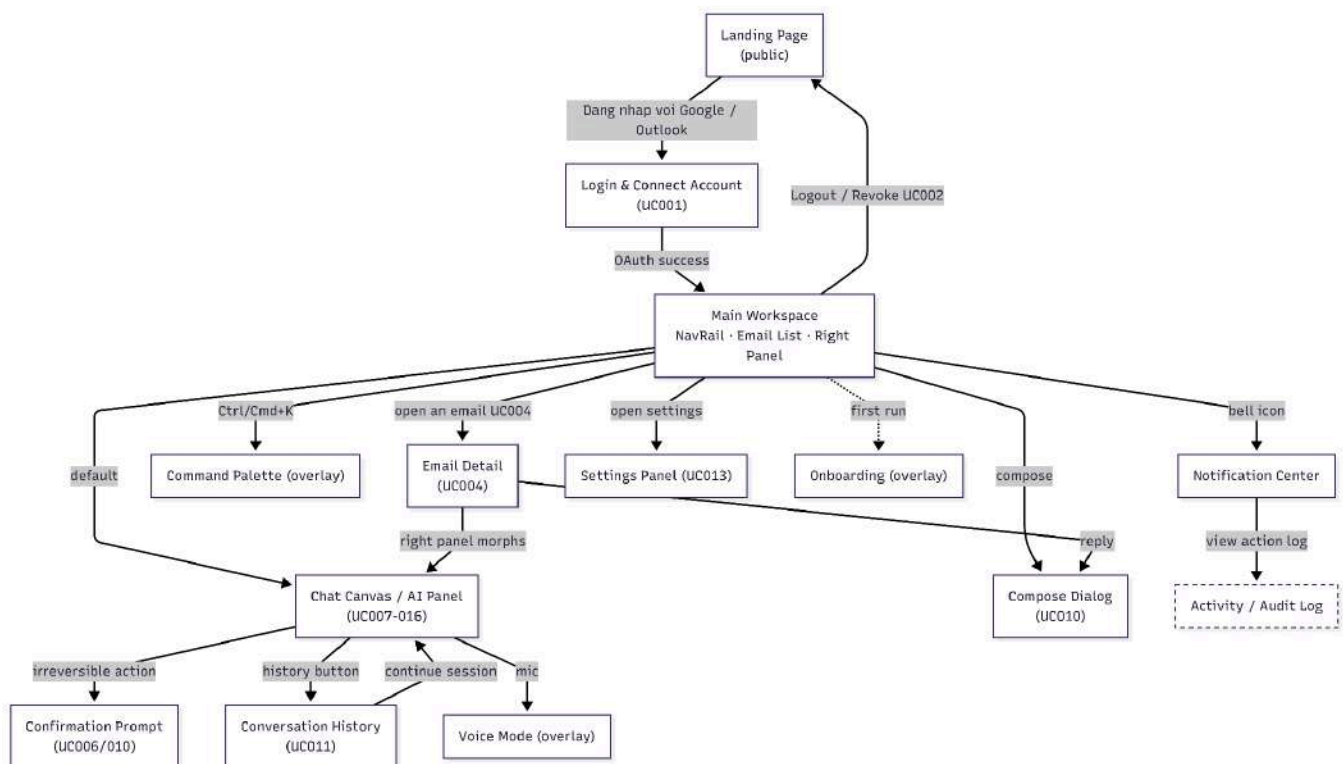
Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by: 24127226 - Phạm Trần Anh Quân

Edited by: 24127250 - Phan Quang Tiến

Reviewed by: 24127529 - Nguyễn Chí Tài

[Draw a screen diagram illustrating the relationships and transitions between the screens]



[List the screens]

Seq	Screen	Description
1	Landing Page	Public marketing page and the first screen an unauthenticated visitor sees. Presents the value proposition, an overview of the assistant's capabilities, the seven email categories, the subscription plans and their AI processing time windows, and the sign-in call to action.
2	Login & Connect Account	Google OAuth 2.0 sign-in + Gmail consent; entry point for all use cases (UC001).
3	Main Workspace	Three-column shell — NavRail (folders) · Email List (center) · Right Panel (right); hosts UC003–UC016.
4	Email List	Card list with search, 7 category filters, quick filters, multi-select bulk actions (UC003/005/006).
5	Email Detail	Full email (sender/recipients/body/attachments) + thread smart card + contextual agent actions (UC004/008/016).
6	Chat Canvas (AI Panel)	Default right panel; NL chat rendered as text / plan / confirmation / draft / result / generative widgets (UC007–016).
7	Confirmation Prompt	In-canvas human-in-the-loop Approve/Reject card for irreversible actions (UC006/010).
8	Compose Dialog	AI-assisted compose (ghost-text) with confirmation before sending (UC010).
9	Conversation History	Slide-in drawer: view/continue/search/pin/rename/delete saved chat sessions (UC011).
10	Settings Panel	Modal: theme, language, MCP access, logout & revoke Gmail (UC013/UC002).

11	Command Palette	⌘K overlay for quick NL commands, navigation, theme switch.
12	Voice Mode	Mic overlay: STT → agent, TTS read-back (extends UC007).
13	Onboarding	First-run coachmark (4 tips), shown once.
14	Notification Center	Surfaces urgent emails / task reminders; matches the architecture diagram.

1.7 Screen Specifications

[Students should select and present the specifications of a few (5–6) of the most important screens. For the remaining screens, only the user interface design needs to be drawn]

Please include the ID and full name of the student who authored, reviewed, or updated this section.

Written by: 24127226 – Phạm Trần Anh Quân

Edited by: 24127250 – Phan Quang Tiến

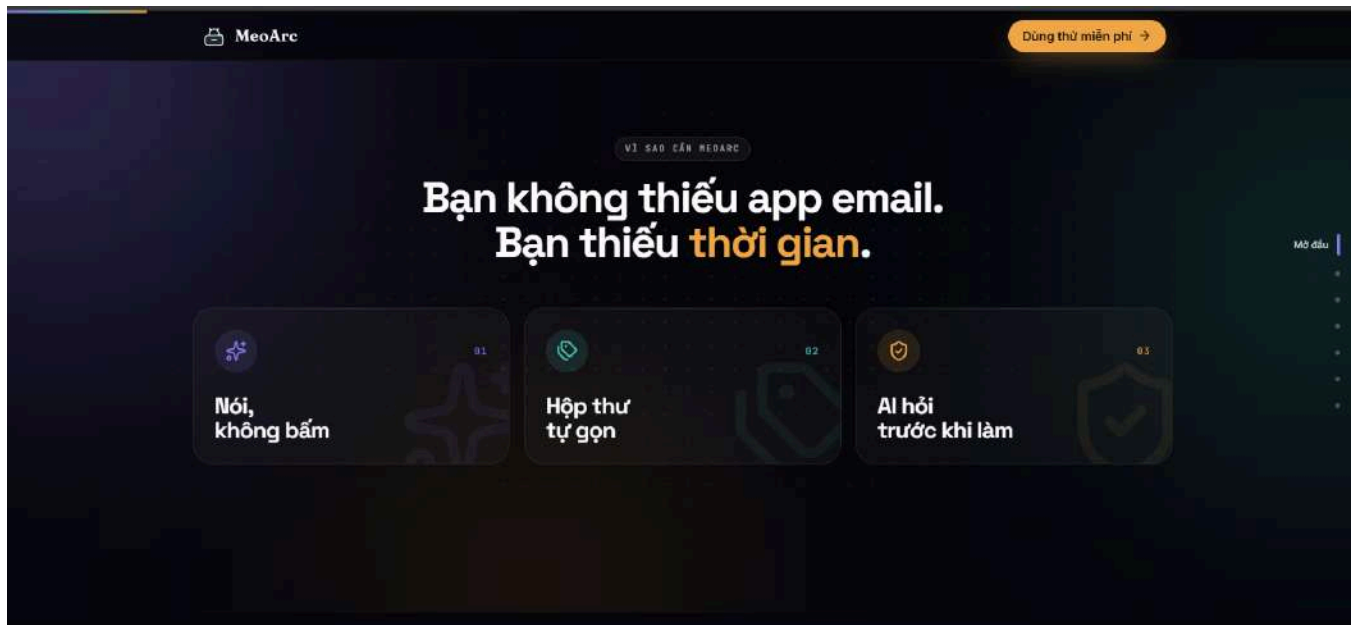
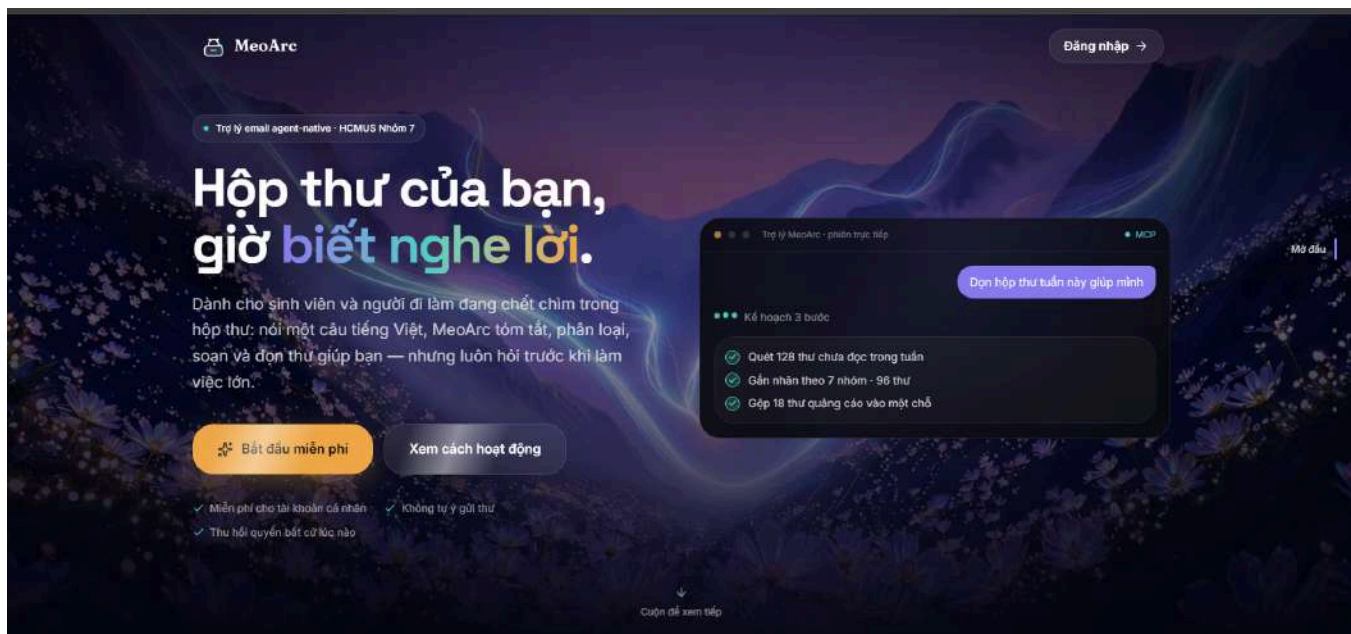
Reviewed by: 24127529 – Nguyễn Chí Tài

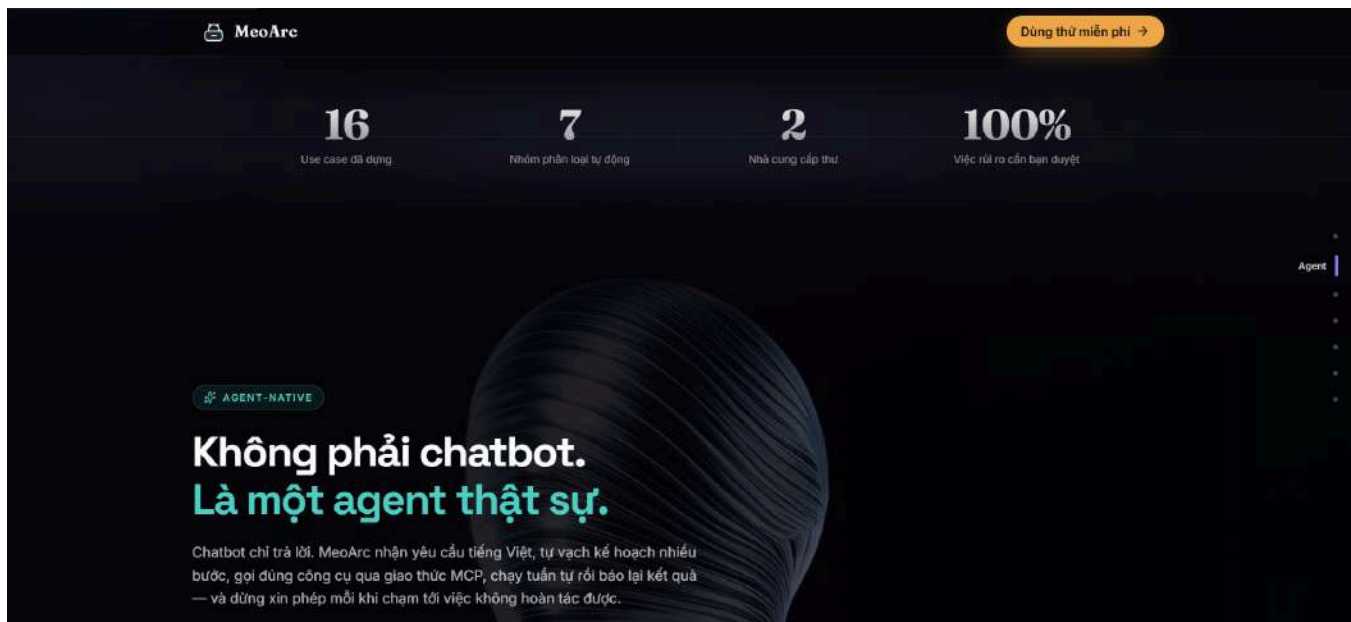
1.7.1 Screen "Landing Page"

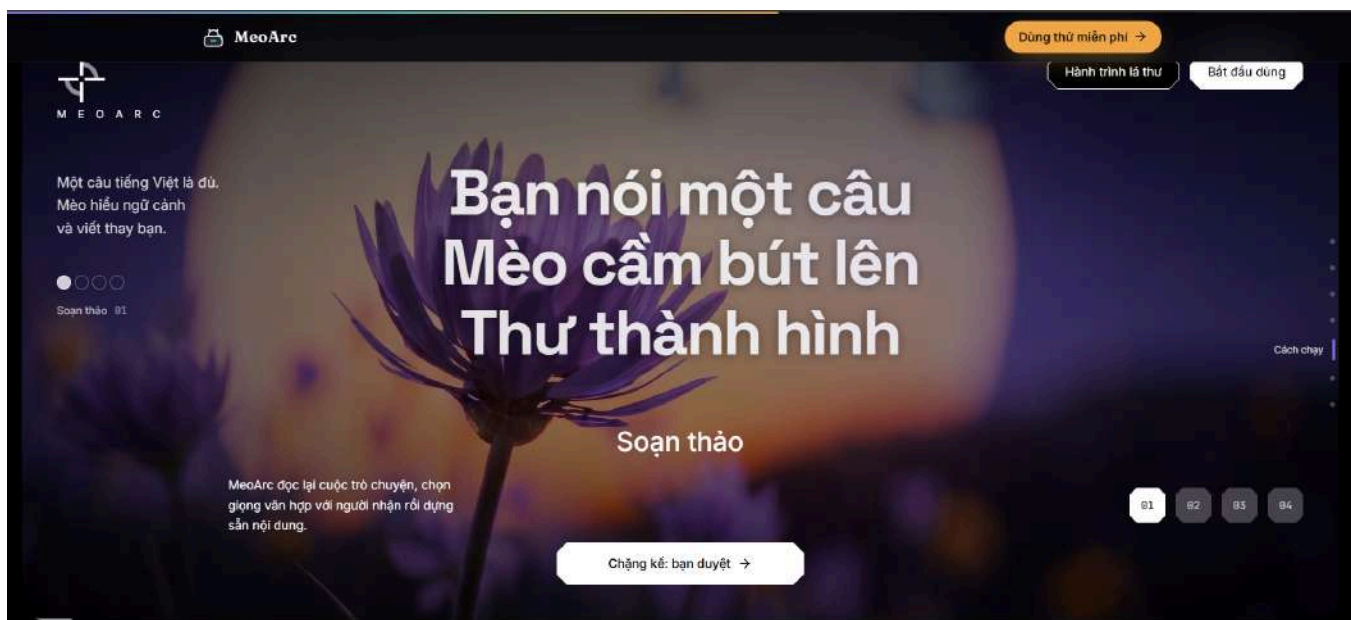
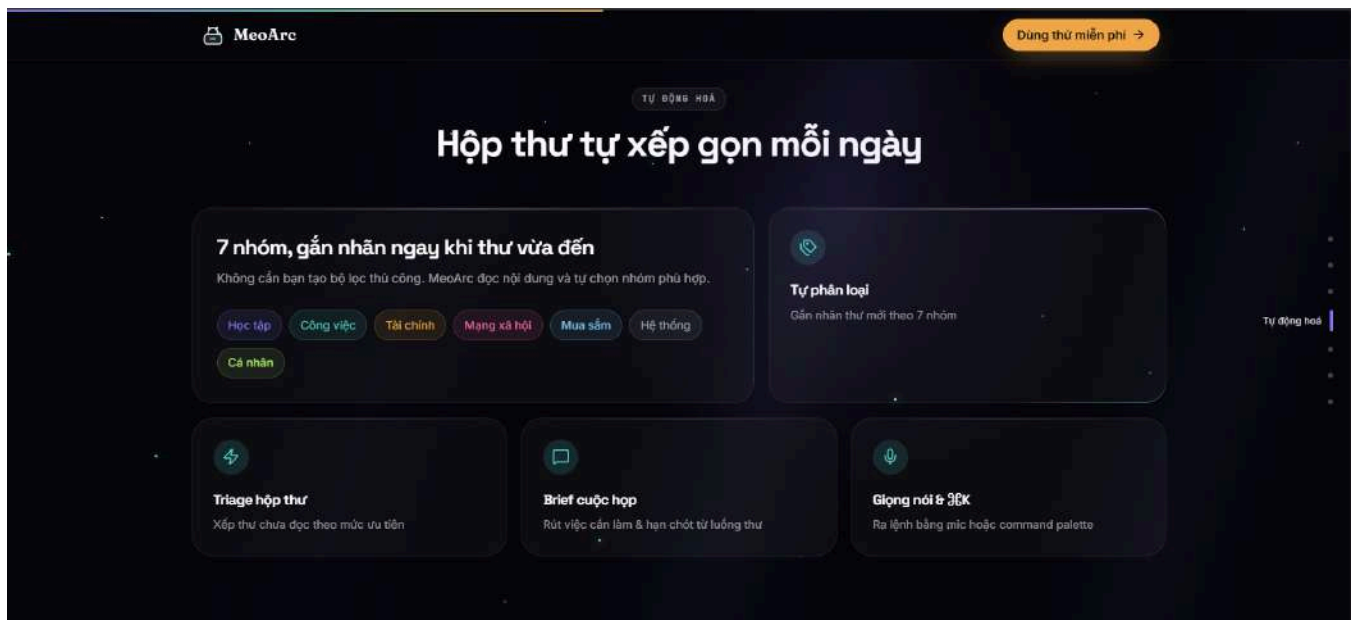
A full-width marketing page, and the first screen an unauthenticated visitor sees. It is organised as a vertical sequence of sections: a hero section with a full-bleed background video, the product name and value proposition, and a primary call-to-action; a statistics band with animated counters; a section presenting the AI assistant over a full-bleed background video; a feature grid in bento layout, icon-led rather than text-led; a second bento grid presenting the seven email categories (School, Work, Finance, Social, Shopping, System, Personal) each in its own colour; a narrative section illustrating the journey of an email, driven by a video whose playback position follows the scroll position; a marquee of the technologies used; a frequently-asked-questions list; a closing call-to-action over a background video; and a footer. A fixed navigation bar stays visible while scrolling and contains anchors to Features, How it works, Pricing and FAQ, together with the sign-in button and a light/dark theme toggle.

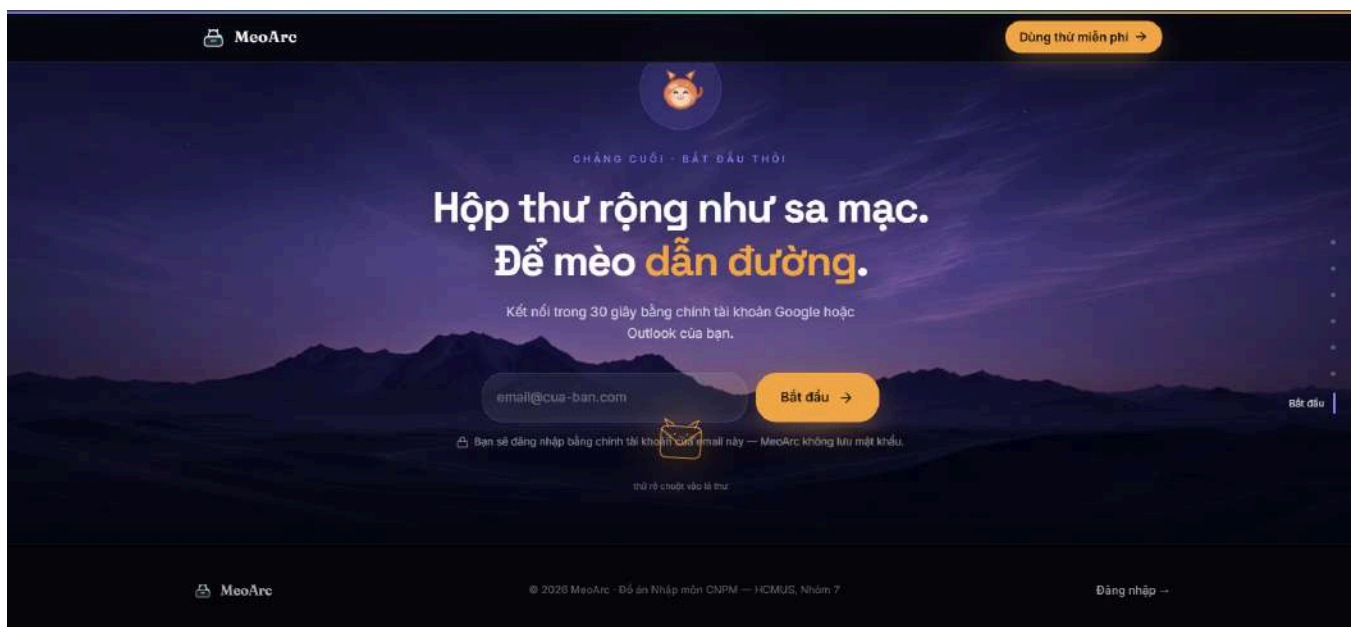
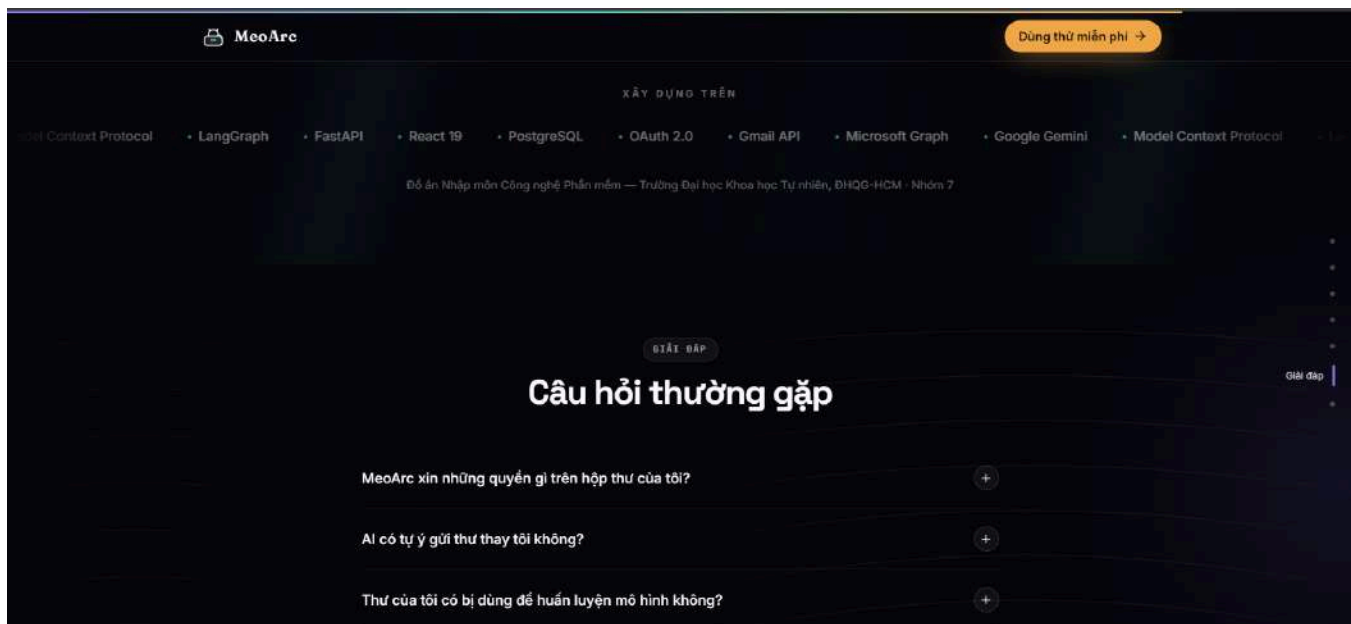
Event handling. Clicking "Đăng nhập với Google" or "Đăng nhập với Outlook" starts the OAuth 2.0 flow described in UC001 and, on success, navigates directly to the Main screen – the

visitor does not return to the Landing page. Clicking a navigation anchor scrolls smoothly to the corresponding section. Clicking "**Gói dịch vụ**" opens the subscription plan comparison, which presents the Free, Pro and Pro Max tiers together with the AI processing time window of each (90, 180 and 365 days respectively). The theme toggle switches Light/Dark instantly and requires no sign-in. Scrolling drives the section reveal animations and the playback position of the email-journey video. A visitor who is already authenticated and opens the Landing page is redirected to the Main screen; signing out from the Main screen returns the user to the Landing page.





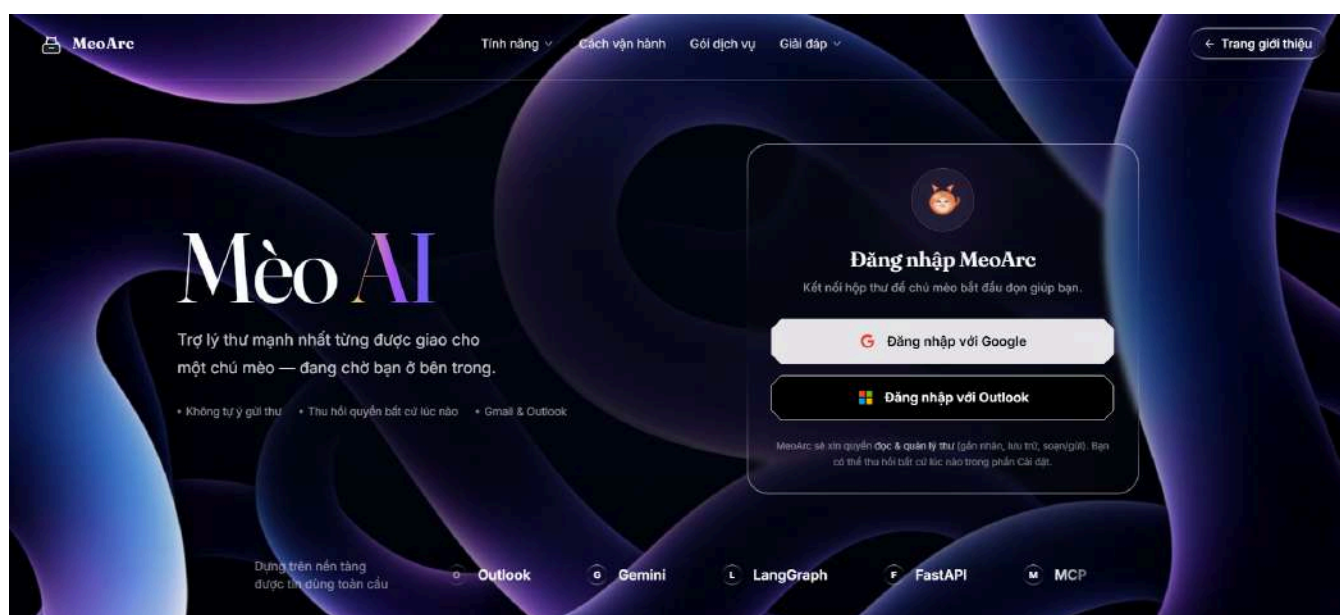




1.7.2 Screen “Login & Connect Account”

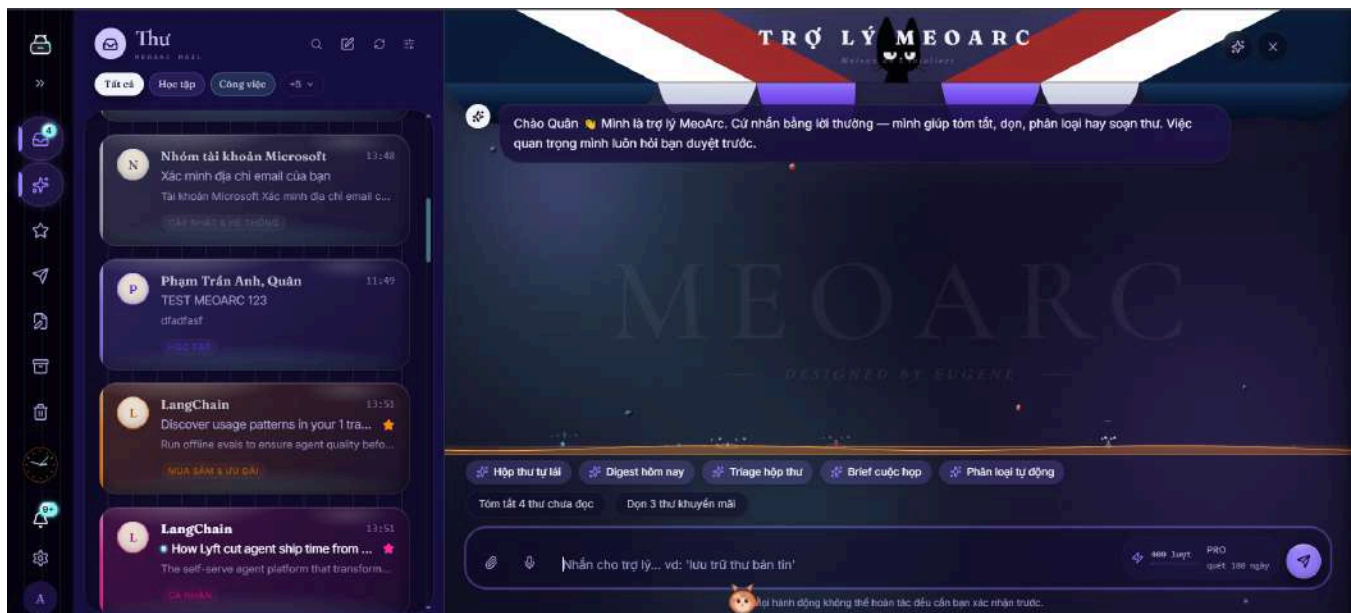
Presentation format. A single centered screen over a themed background image (light/dark variants). The main block contains: the MeoArc logo and tagline, three reassurance lines ("important actions always require your confirmation", "secure connection via your Google account"), one CTA button "Đăng nhập với Google", and a scope note explaining that MeoArc

requests permission to read & manage mail, revocable at any time in Settings. A theme toggle sits in the corner. Event handling. Click "Đăng nhập với Google" → starts the OAuth 2.0 flow; on success the system creates/restores the users record → navigates to the Main screen (UC001). First-time sign-in provisions the account automatically. OAuth error / user cancels → stays on Login with a friendly error message. Theme toggle → switches Light/Dark instantly, no sign-in required.



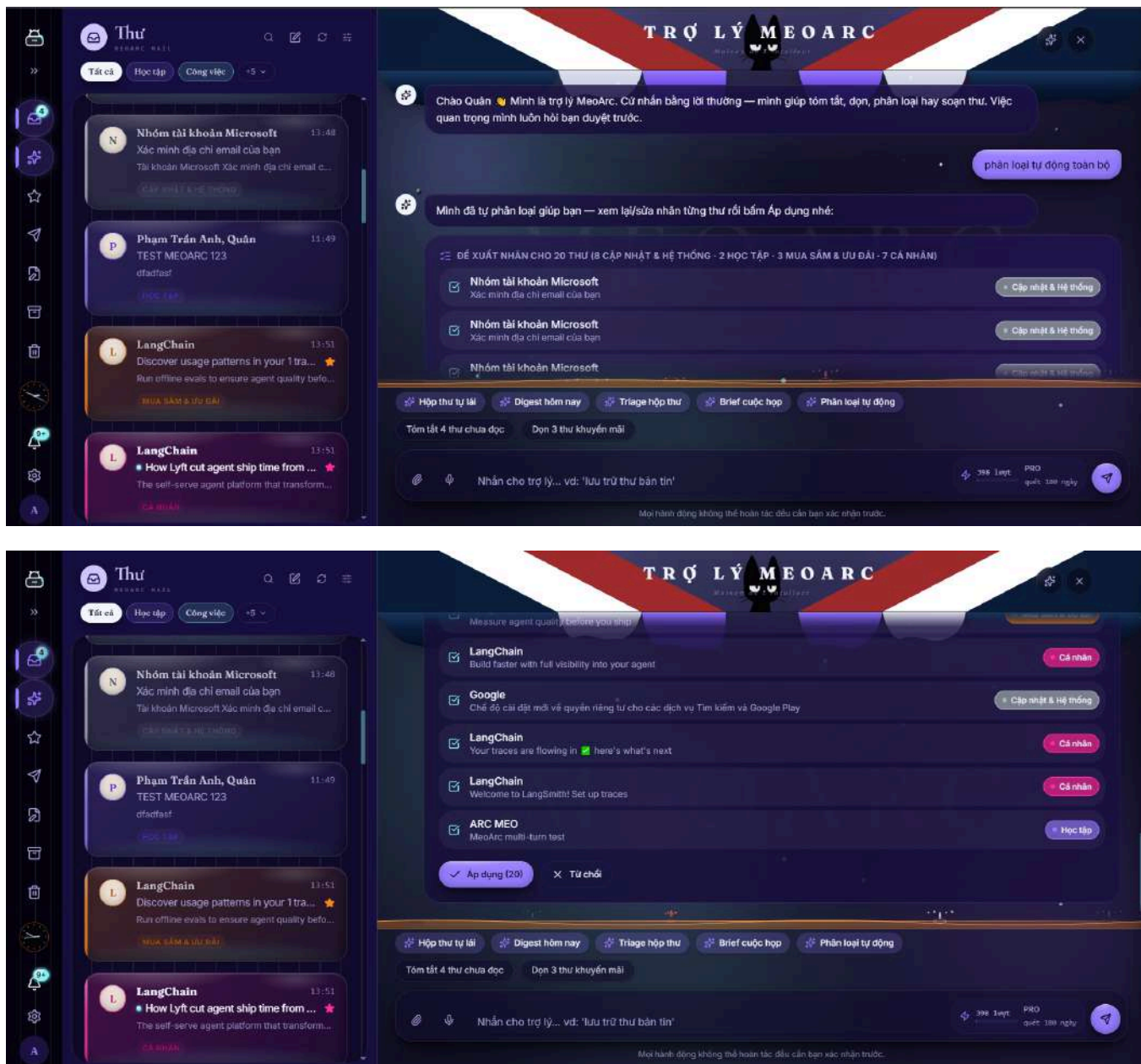
1.7.3 Screen “Main screen (Chat + Dynamic Canvas)”

The Main screen (Chat + Dynamic Canvas) uses a three-column layout with a navigation sidebar on the left, an email list in the center, and an AI chat panel on the right. The sidebar lets users switch between inbox, sent, drafts, and other folders, while the email list displays message cards with sender, subject, and preview. When the user types a request into the chat input and presses Send, the AI assistant analyzes the command and responds with the appropriate result – text, an actionable plan with Approve and Reject buttons, a draft email with Send and Edit options, or an interactive summary card for digests, triage, and meeting briefs. Clicking Approve on a plan executes the steps automatically, while clicking Send on a draft triggers the actual email dispatch. The right panel smoothly transitions between the chat canvas and the email detail view, and additional tools like voice input, command palette, and conversation history are accessible from the chat header.



1.7.4 Screen “Confirmation Dialog (Human-in-the-loop)”

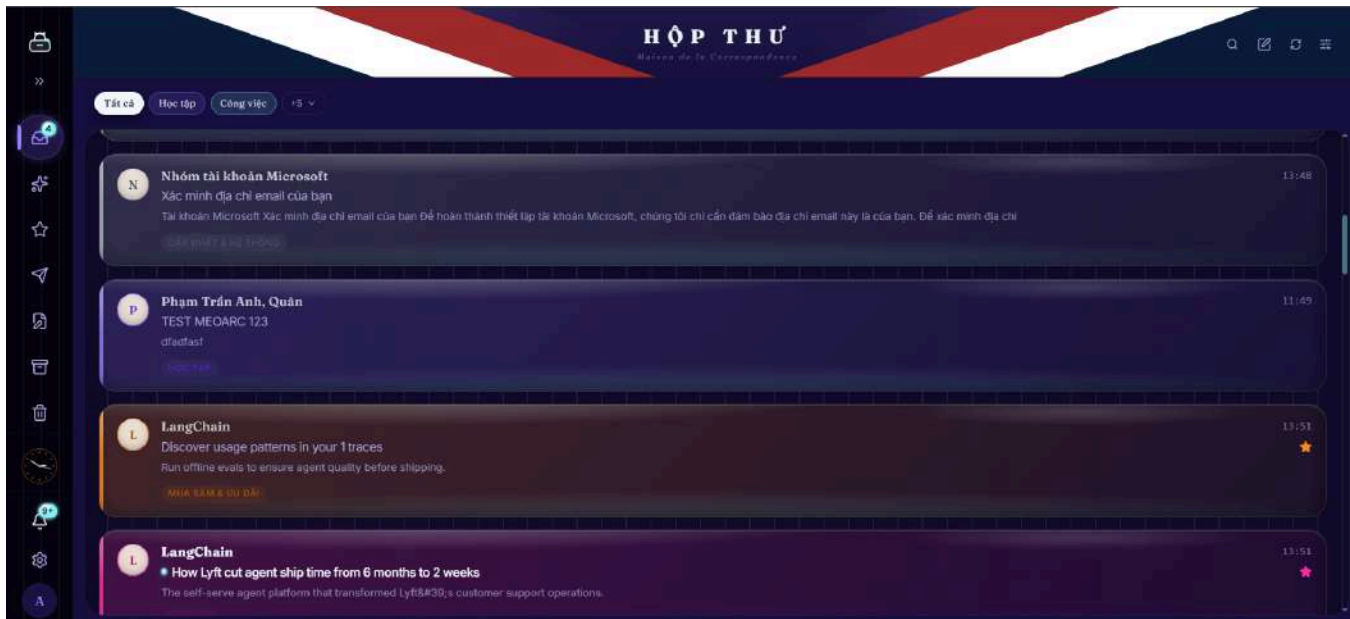
The Confirmation Dialog (Human-in-the-loop) appears as an embedded card inside the AI chat panel whenever the agent proposes a destructive or irreversible action such as deleting emails, sending a draft, applying labels, or executing multi-step bulk operations. The card displays a clear title (“Kế hoạch đề xuất” for plans or a draft preview for emails), a numbered step list with dashed connectors, and a warning banner if the action is risky (e.g., permanent deletion). Two prominent action buttons – “Approve” (primary highlighted) and “Reject” (outlined) – sit at the bottom of the card, demanding explicit user consent before any backend execution occurs. When the user clicks Approve, the agent executes the steps sequentially with animated progress indicators, and a completion confirmation replaces the card upon success. When the user clicks Reject, the plan is discarded, and the agent acknowledges the cancellation. While a confirmation is pending, the card is highlighted with a spotlight effect and all other messages are dimmed to focus the user's attention on the decision.



1.7.5 Screen “Email List”

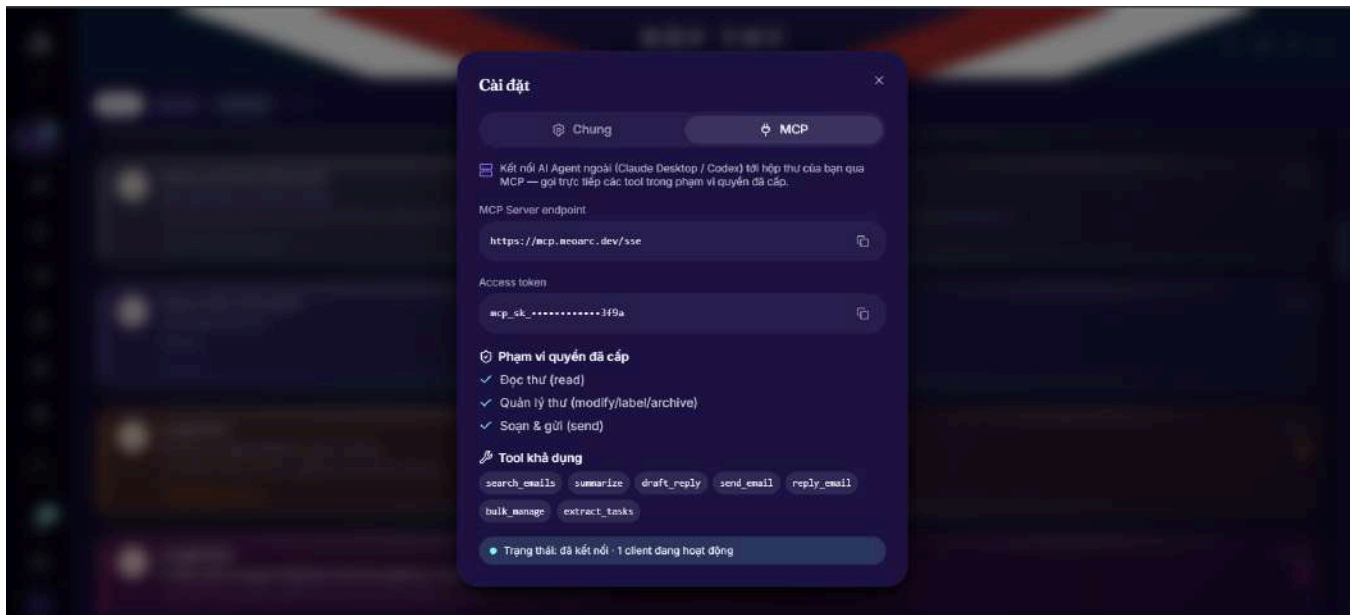
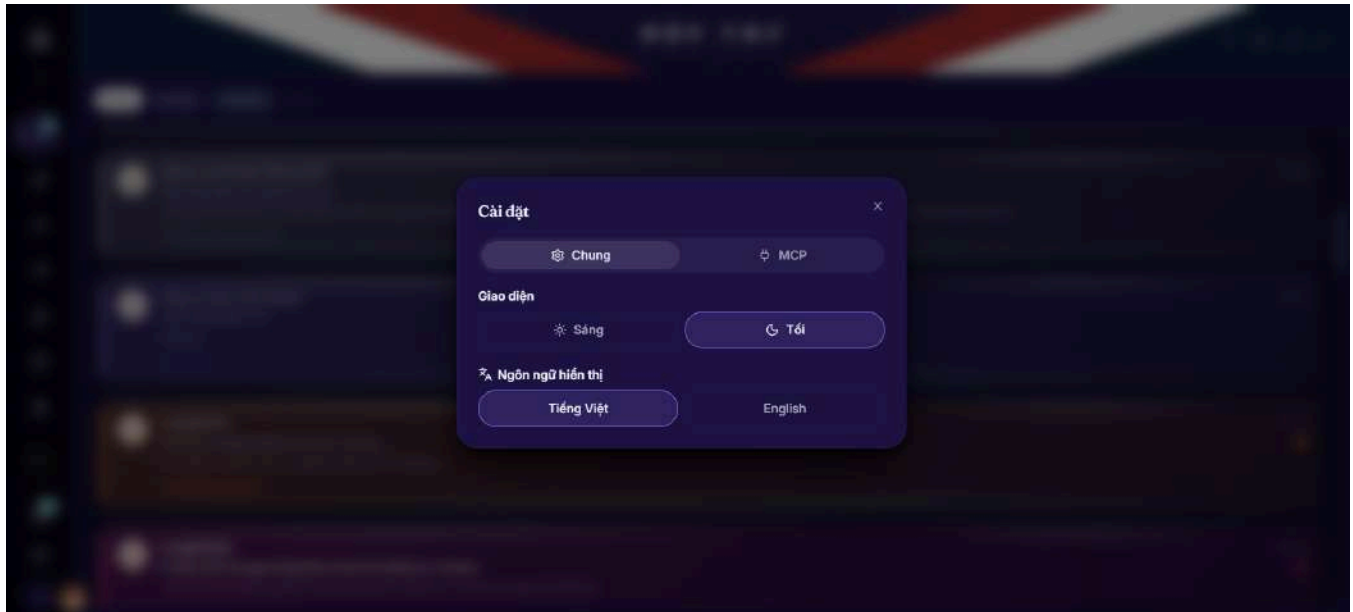
The Email List screen occupies the center column between the navigation sidebar and the AI chat panel, featuring a resizable layout (300–560px) that can be dragged by its right-edge grip handle. A header bar displays the current folder title (“Inbox”, “Sent”, etc.) alongside buttons for toggling the search field, composing a new email, refreshing the list, and opening quick filters. Below, category tag chips (7 color-coded categories plus “All”) let users filter emails by topic, while

expandable quick-filter toggles narrow results further by unread, starred, or attachment status. Each email appears as a card showing the sender name, subject line, preview snippet, timestamp, an unread dot, a star icon, and optional priority or label badges. When the search bar is opened via its toggle or by pressing /, users can type keywords or toggle natural language mode (sparkle icon) to query in plain Vietnamese. Clicking any email card opens it in the right panel, the checkbox on hover enables multi-select with a floating toolbar for batch marking, labeling, or deleting, and pressing j/k navigates cards via keyboard while Enter opens the highlighted one.



1.7.6 Screen “Settings Panel”

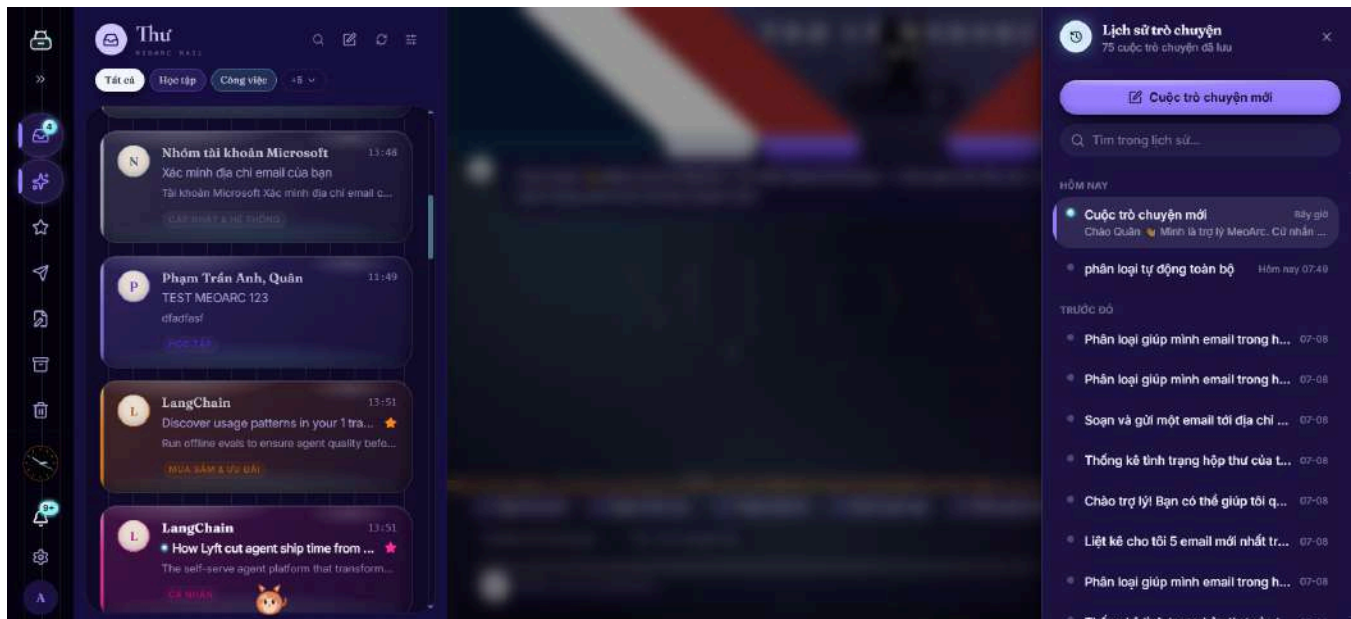
The Settings Panel opens as a modal dialog triggered by the gear icon at the bottom of the navigation sidebar. Two tabbed sections — ”General” and ”MCP” — organize the available options at the top of the dialog. Under the General tab, users can switch between Light (”Sáng”) and Dark (”Tối”) themes via side-by-side buttons, and toggle the display language between Vietnamese and English, with the selection immediately saved to local storage. The MCP tab displays connection details for external AI agents (Claude Desktop, Codex), including a copyable server endpoint and access token, along with a list of granted permission scopes (read, modify, send) and all available MCP tools such as search, summarize, draft reply, send email, and bulk manage. A status indicator at the bottom confirms the MCP connection state and active client count.



1.7.7 Screen “Conversation History List”

The Conversation History List is a slide-in drawer that opens from the right edge of the screen, triggered by the sparkle button in the ChatPanel header. It displays all saved chat sessions grouped into sections â ” pinned conversations at the top, followed by time-bucketed groups

(Today, Yesterday, Last Week, Older). Each session entry shows the conversation title, a truncated preview of the last message, a timestamp, and a pin indicator; the currently active session is highlighted with a left accent bar. Hovering reveals a toolbar with three icon buttons: Pin/Unpin to keep important conversations accessible, Rename to edit the title inline, and Delete which expands a destructive confirmation row with Cancel and Delete buttons. A "New Conversation" button at the top instantly creates a fresh session, and a search field with a magnifying glass icon lets users filter sessions by keyword across titles and preview text. Pressing Escape closes the drawer, and clicking any session entry switches the chat panel to that conversation, loading its full message history from the backend if needed.



6

AI Usage Declaration

Based on the provided AI Usage Guideline document, teams must submit their AI usage declaration here. Students are encouraged to leverage AI tools creatively and effectively in this project.

ChatGPT. GPT-4o, chatgpt.com, accessed 19:59 on June 17, 2026, summarize the chat: “Learn about apps that support drawing EER for conceptual modeling, understand the concept, and how to accurately draw according to the assignment requirements.” <[Đoạn chat với GPT](#)>

ChatGPT. GPT-4o, chatgpt.com, accessed 14:59 on July 22, 2026, prompt: “với phần báo cáo: Screen Specifications, yêu cầu Clearly describe the presentation format and the handling of each event within the screen. Include a design image of each screen. Thì tôi nên trình bày theo format nào cho chuẩn. cho tôi 1 example đơn giản với màn hình đăng nhập nói chung đi.”, the LLM responds a format that is too specific <[Dialogue](#)>

ChatGPT. GPT-4o, chatgpt.com, accessed 15:00 on July 22, 2026, prompt: “nhưng mà rõ ràng đề yêu cầu 5-6 câu mô tả compact thôi mà.”, the LLM responds a compact format and an clear example <[Dialogue continue](#)>

7 Presentation

Teams are required to record presentation videos of their work using the provided template.

Every team member must participate in the presentation, specifically covering the sections their contributed to the project.

Videos must not exceed 30 minutes.

Please upload your videos to Youtube (set to either Unlisted or Public).

Provide the Youtube links the the field below.

[GROUP 7 _ DESIGN](#)

8

Reflective Report

Which sections of this template are most helpful for implementation, and which are unnecessary? Provide specific examples to support your reasoning.

Written by: 24127250 - Phan Quang Tiến

Edited by: 24127545 - Nguyễn Ngọc Thiên

Reviewed by: 24127226 - Phạm Trần Anh Quân

Most useful sections during implementation

The **Data Diagram** and **Data Specification** were the two most useful sections for the team during actual implementation. Being required to explicitly list every entity, attribute, data type, and constraint before writing any code helped the team catch design issues early – for example, when defining the **Emails – Embeddings** relationship as 1-to-1, the team realized a **UNIQUE** constraint was needed on the **email_id** column in the **Embeddings** table, rather than simply adding a foreign key. Without this specification step, that design flaw would likely have only surfaced after the migration script was already written, costing extra time to fix.

The **Conceptual Model (EER Diagram)** also played an important role, as it forced the whole team to agree on the data semantics before splitting up individual coding tasks. For instance, clearly defining **ConnectedAccounts** as a separate entity from **Users** (instead of merging the tokens into the **Users** table) made it much easier to later extend the "connect multiple Gmail accounts" feature without having to rework the schema.

Sections that were less necessary or could be shortened

Class Specifications (requiring detailed specs for the 9-10 most important classes) turned out to be fairly time-consuming relative to the value it added, for a project of this course's scale. Most of

that information (attributes, visibility, methods) can already be inferred directly from the code when building with an AI-agent-oriented approach using modern frameworks, where classes tend to be thin and mostly consist of orchestration logic that calls tools/APIs. The team suggests that for projects with an agent/service-oriented architecture (like MeoArc), this section should focus on specifying the main services/modules rather than classes in the traditional OOP sense.

The level of detail required in the **Screen Diagram** and **Screen Specifications** (describing every event handled on each screen) overlapped considerably with the UI/UX design work already done in Figma or other prototyping tools beforehand. Having to rewrite in text what was already clearly shown in the prototype felt redundant to the team, adding little new value relative to the effort required.

Conclusion

Overall, the sections directly related to **data (Data Design)** and **system architecture (Architectural Design)** provided the highest practical value, since they form the foundation that directly affects the coding process and are difficult to fix once mistakes are made. In contrast, the detailed specification sections for UI or classes could be shortened or replaced with links to the prototype/design tool, allowing the team to spend more time on design decisions with greater overall impact.